

GenOS (생성형 AI 플랫폼)

Genos 문서 전처리

교육 자료

목차

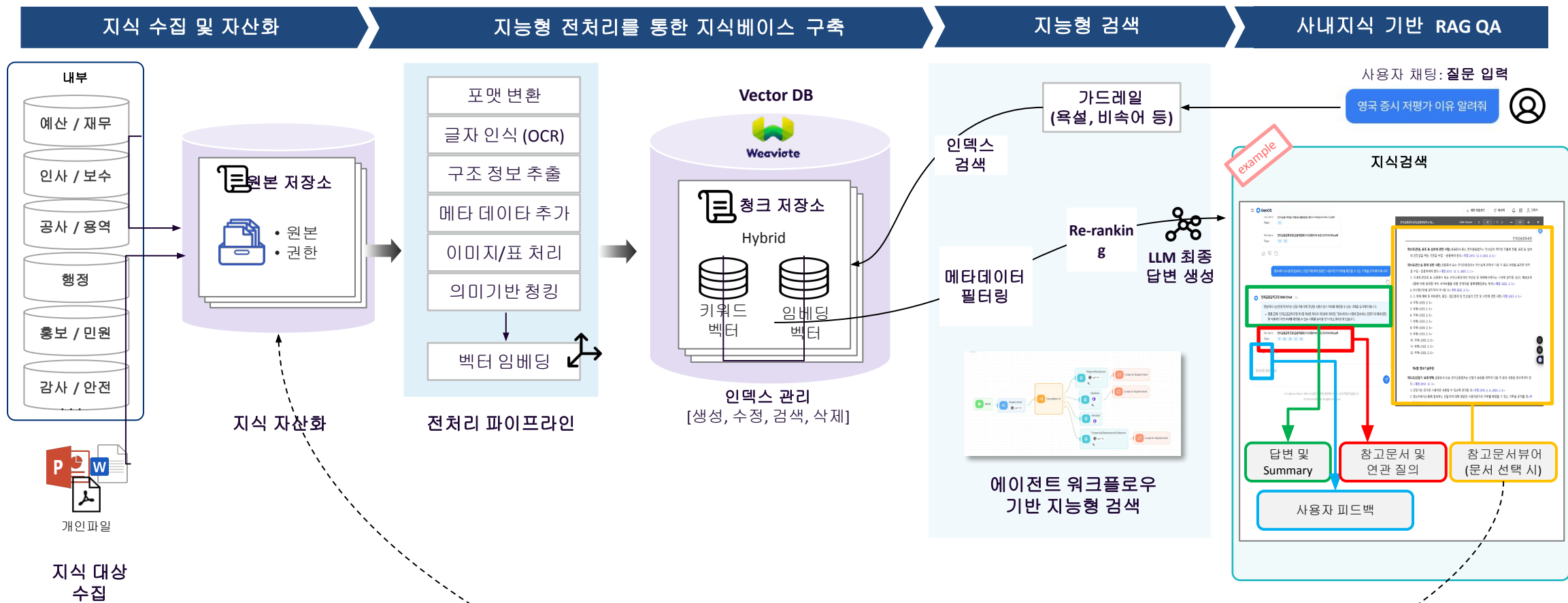
1. 전처리 개념 설명
2. 전처리 별 문서변환/전처리 옵션 도해
3. 전처리 프로젝트 소스 사용방법
4. GenOS 전처리 사용방법
5. 기타

1. 전처리 개념 설명

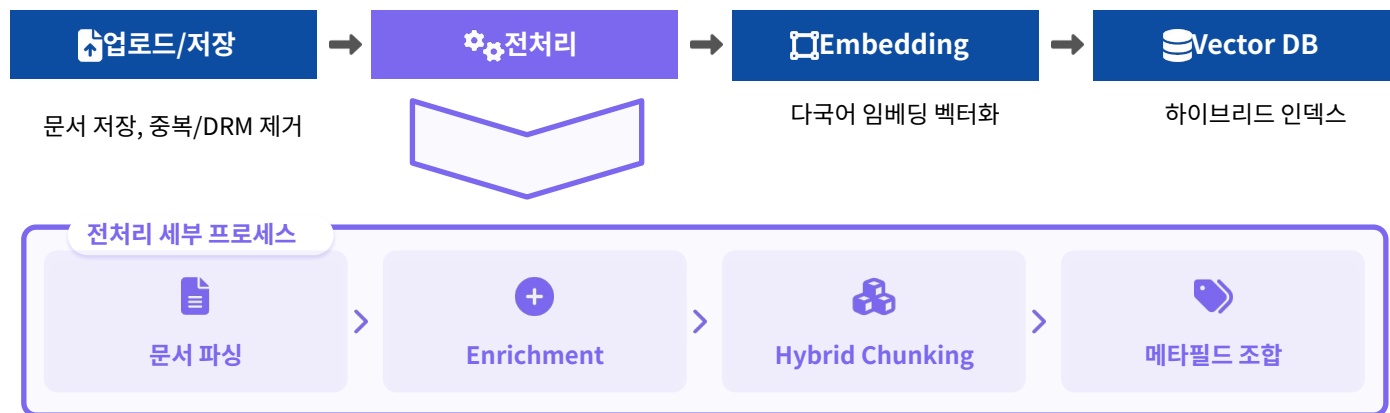
Genos 데이터 파이프라인 구성 요소

- 개요
 - RAG 시스템 구성/파이프라인 플로우(요약)
- 파이프라인 세부 플로우
 1. 업로드/저장:
 - (첨부용 vs 적재용) 업로드 파이프라인 차이점
 - (첨부용 vs 적재용) 데이터원천 차이점
 2. 전처리:
 - a. (지능형 전처리) 문서 파싱 : AI 모델을 사용한 파싱
 - b. (지능형 전처리) Enrichment: ToC 생성, 작성일 추론, 이미지 설명 생성
 - c. (첨부용 vs 적재용) Chunking 방식의 차이점
 3. Embedding
 4. Vector DB 하이브리드 검색
- 전체 요약

[개요] RAG 시스템 구성



[개요] 파이프라인 플로우(요약)



4단계 파이프라인 구성

1. 업로드/저장

- 첨부용: 사용자가 챗봇에 임의 파일 업로드, 즉시 처리
- 적재용: 데이터레이크에서 일괄 수집, 품질 중시
- 공통: 중복/DRM 제거, 메타데이터 추출, 파일 정규화

2. 전처리 (4단계 세부 프로세스)

- 문서 파싱: 형식별 텍스트 추출, OCR, 레이아웃 분석
- Enrichment: ToC 생성, 작성일 추론, 이미지 설명 생성(적재용 전용)
- Hybrid Chunking: 레이아웃/의미 기반 분할 (적재용 전용)
- 메타필드 조합: 추출 메타데이터와 청크 연결

3. Embedding

- 다국어 임베딩 모델로 텍스트 필드 벡터화

4. Vector DB

- Dense + Keyword 하이브리드 인덱스 구성
- 가중 결합($\alpha \cdot \text{dense} + (1-\alpha) \cdot \text{keyword}$) 검색 지원
- 메타필터 및 재순위 기능 제공

[세부 플로우] 1. 업로드/저장

- (첨부용 vs 적재용) 업로드 파이프라인 차이점

첨부용 파이프라인

- 실시간 문서 업로드 (사용자 챗봇 질의)
- 빠른 처리 속도 우선 (지연 최소화)
- 임시 저장 (세션 기반 단기 보관)

적재용 파이프라인

- 일괄 수집 (데이터레이크 문서 자동화)
- 품질 및 정확도 중심 처리
- 영구 저장 (벡터 DB 기반 장기 보관)

비교 항목	첨부용	적재용
처리 시간	빠름	느림
메타데이터	최소 필수	포괄적, Enrichment 통한 추가 생성
저장소	임시(24시간)	영구
재처리	불가	가능

메타데이터 수집

문서 속성 추출 및 기본/확장 메타 생성

전처리 큐 전달

문서ID 기반 작업 단위화 및 우선순위 설정

업로드/저장 핵심 단계

1. 중복 해시 체크

- SHA-256 해시 알고리즘으로 문서 중복 여부 확인
- 첨부용: 임시 해시 저장소 활용 / 적재용: 메인 해시 DB 연동

2. DRM/암호 제거

- 보안 해제 후 표준 포맷으로 정규화
- 첨부용: 간소화된 변환 / 적재용: 완전한 정규화

3. 메타데이터 수집

- 소스, 작성자, 수집일, 문서ID, MIME 타입 등 추출
- 첨부용: 제한된 필수 메타 (파일명, 크기, 타입)
- 적재용: 포괄적 메타데이터 수집, Enrichment를 통한 추가 생성

4. 전처리 큐 전달

- 문서ID를 기준으로 작업 단위화하여 등록
- 첨부용: 실시간 고우선순위 큐
- 적재용: 일괄 처리 큐 (품질 최적화)

문서 처리 워크플로우

- 첨부용: 사용자 업로드 → 임시저장 → 빠른 텍스트 추출
- 적재용: 스케줄 수집 → 영구저장 → 품질중심 처리
- 메타데이터 기반으로 후속 전처리 과정 최적화

[세부 플로우] 1. 업로드/저장

- (첨부용 vs 적재용) 데이터원천 차이점

Genos 데이터 파이프라인은 총 5가지 유형의 데이터 원천을 처리 - 적재용(3종)과 첨부용(2종)으로 구분
이는 파이프라인 흐름에 중요한 영향을 미침

 첨부용 원천 (기본형 전처리)

1. 사용자 첨부 파일

이미지, PDF(스캔 포함), Office(PPTX, DOCX, XLSX), HWP/HWPX 등 다양한 형식의 문서
2. 경량 텍스트

Markdown, 웹문서 등 단순 구조의 경량 텍스트 소스

 적재용 원천 (지능형 전처리)

1. 일반 PDF

외부 출판물, 연구 보고서 등 표준 PDF 문서
2. 규정 문서 (PDF)

조항/호/목 인지가 필요한 특수 규칙 적용 문서
3. 기타(JSON+PDF)

한국은행 독자적 BOK 지식 스키마 기반 문서 (T-Drive 최종본)

 첨부용 원천 처리 특징

사용자 챗봇 질의 중 첨부된 임시 문서에 대한 빠른 분석이 목적
간소화된 구조 분석과 빠른 처리 속도가 중요

- 간소화된 구조 추출:
- DOCX, HWP/HWPX → Markdown 형태 추출
 - PDF → 텍스트만 추출
 - 그 외 → PDF 변환 후 텍스트 추출
 - Enrichment 미적용
 - Max token + overlap 기준 단순 분할 (문맥 미고려)
 - 페이지/BBox 등 메타데이터 미추출

 적재용 원천 처리 특징

데이터레이크의 대량 문서를 정확하게 구조화하여 지속적 검색을 위한 고품질 색인을 생성. 품질과 정확도를 중시.

- 내부 문서(JSON+PDF): BOK 지식 스키마와 PDF 페어로 저장된 문서로 메타데이터가 풍부
 - 일반 PDF: 텍스트 레이어 유무에 따른 OCR 처리와 완전한 레이아웃 분석이 중요
 - 규정 문서: 계층적 구조(조항/호/목) 파악이 중요하며 전용 전처리기를 사용
- 4단계 전처리 적용:
(1) 문서 파싱 → (2) Enrichment → (3) Hybrid Chunking → (4) 메타필드 조합
- ToC 생성, 작성일 추론, 이미지 설명 생성 등 Enrichment 기능 사용
 - 의미/레이아웃 기반 Hybrid Chunking으로 문맥 보존
 - 상세 메타데이터 추출 및 청크와 연결

[세부 플로우] 2. 전처리

- (첨부용 vs 적재용) 전처리 파이프라인 차이점



⚡ 첨부용 전처리

사용자 첨부 파일 빠른 처리 최적화

DOCX, HWP/HWPX → Markdown 추출

PDF → 텍스트 직접 추출

그 외 → PDF 변환 후 텍스트 추출

토큰 기반 단순 분할 (max token + overlap)

⚙️ 적재용 전처리

데이터레이크 문서 구조화 파이프라인

OCR, Layout, Table, Reading Order 모델 활용

ToC 생성, 작성일 추론, 이미지 설명 생성으로 문맥 강화

의미 기반 Hybrid Chunking으로 문맥 보존

상세 메타데이터 추출 (BBox, 페이지, 유형)

[세부 플로우] 2. 전처리

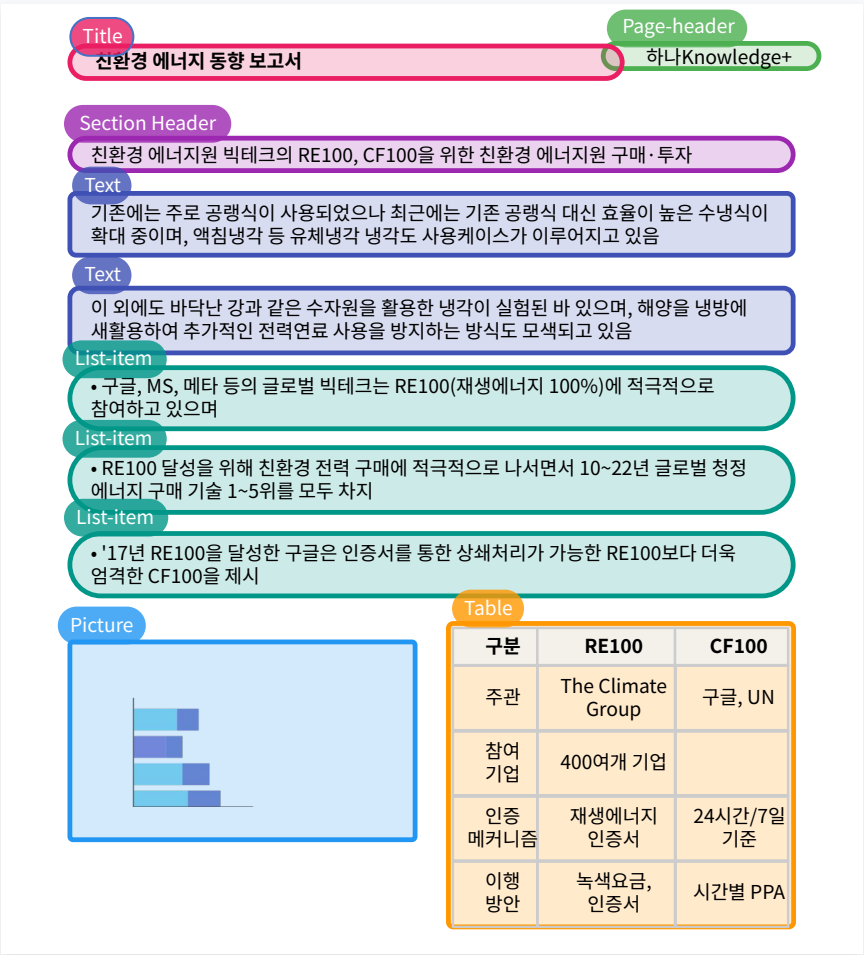
a. (지능형 전처리기) 문서 파싱 : AI 모델을 사용한 파싱

- 4가지 AI 모델 사용

번호	모델 유형	주요 작업	상세 설명	핵심 특징
1	OCR 모델	스캔 이미지 자동 감지 및 텍스트 추출	디지털/스캔 문서 구분 다국어 및 특수 문자 인식 저품질 이미지 텍스트 복원	자동 스캔 감지 및 텍스트 추출 최적화 저해상도 이미지에서도 고품질 텍스트 복원 다국어/특수문자/표/수식 정확 인식
2	Layout 모델	문서 레이아웃 분석 및 객체 탐지	11가지 BBox 유형 분류 다단 레이아웃 정확 인식 겹치는 요소 구조적 해석	11가지 객체 유형 분류 (제목, 본문, 표, 그림 등) 다단 구조, 복잡 레이아웃 정확 인식 한국은행 문서 형식에 특화된 도메인 최적화
3	Table 모델	표 구조 인식 및 셀 정렬	병합 셀 구조 분석 헤더/바디 셀 자동 구분	복잡한 표 구조 및 병합 셀 정확 인식 헤더/바디/주석 자동 구분 처리 표 데이터를 구조화된 형식으로 변환 (markdown)
4	Reading Order 모델	요소 읽기 순서 자동 정렬	페이지 간 컨텍스트 연결 문서 구조와 참조 관계 유지	인간 읽기 순서 기반 문서 요소 순서화 각주, 표 참조 등 문서 내 관계 매핑 페이지 경계를 넘는 컨텍스트 연결

[세부 플로우] 2. 전처리

a. (지능형 전처리기) 문서 파싱 : AI 모델을 사용한 파싱



문서에 대한 Layout 예

Layout 분석: 11가지 BBox 카테고리

BBox 카테고리 (11가지)

- Title: 문서/장/절의 제목
- Text: 일반 본문 텍스트 블록
- Table: 행/열로 구성된 표 구조
- Caption: 표/그림에 대한 설명문
- Footnote: 본문 참조를 위한 각주
- Page-header: 페이지 상단 반복 정보
- Page-footer: 페이지 하단 정보(번호 등)
- Section Header: 소제목, 주제 구분자
- List-item: 글머리 기호 목록 항목
- Picture: 그림, 차트, 다이어그램
- Formula: 수학/통계 수식, 계산식

활용 방식

- 문서의 구조적 계층화로 의미 기반 검색 지원
- Section Header 기반 Chunking의 핵심 데이터
- Reading Order 결정에 기초 정보로 활용
- 표/그림 영역 식별로 Table 추출 모델 및 Image Description 생성 입력 준비
- 검색 시스템에서 요소별 가중치 차등 적용

출력 정보

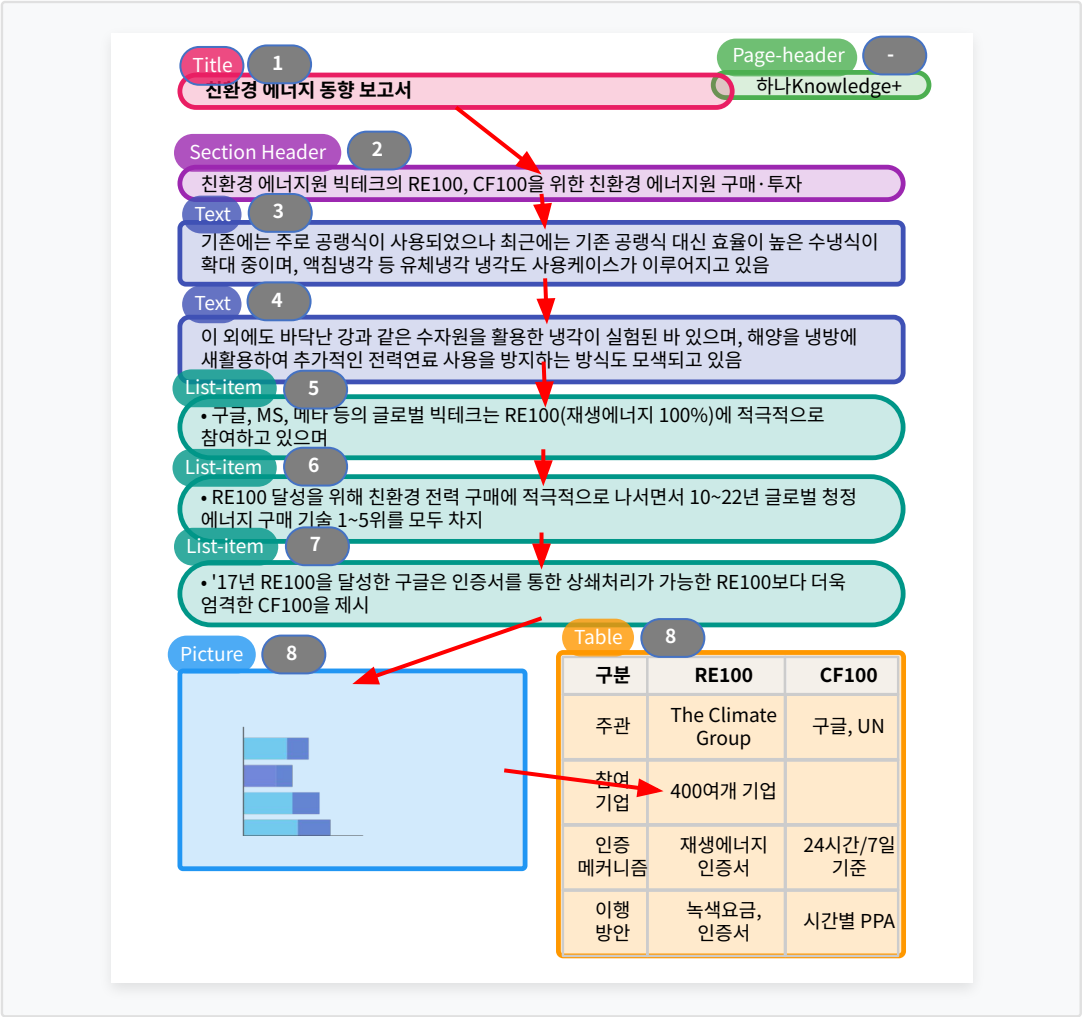
- 요소 위치 좌표 (x, y, w, h)
- 인식 신뢰도 점수 (0~1)
- 페이지 번호와 카테고리
- 요소 간 계층 구조 정보

특징

- 한국은행 문서에 특화된 레이아웃 인식
- 스캔 이미지와 디지털 문서 모두 지원

[세부 플로우] 2. 전처리

a. (지능형 전처리기) 문서 파싱 : AI 모델을 사용한 파싱



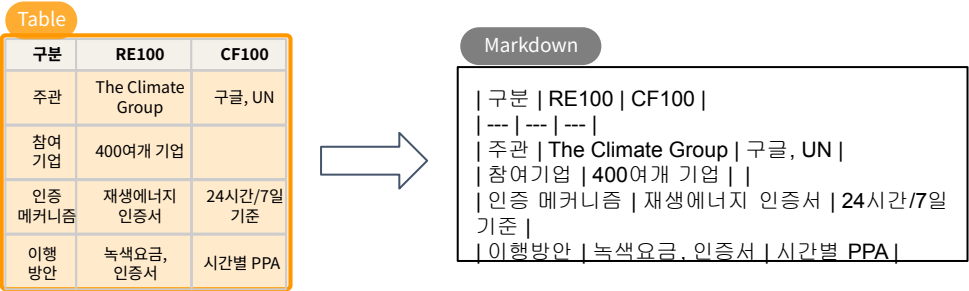
문서에 대한 Layout 예

Reading Order 분석: BBox간의 읽는 순서 추론

활용 방식

Layout의 BBox간의 읽는 순서를 추론.
반복되는 부수 정보(Footnote, Page-header, Page-footer)에는 읽는 순서 부여 안함.

Table 인식: 행/열 구조를 고려한 Table 값 추출



출력 정보

Markdown으로 변환된 Table의 값.
내부 모델이 HTML으로 추출후, Docling을 통한 변환을 거쳐서 최종적으로 Markdown 출력.

OCR 수행: 일반 텍스트에 대한 추출

활용 방식

스캔 이미지에서의 글씨 인식
다국어 및 특수 문자 인식

[세부 플로우] 2. 전처리

b. (지능형 전처리기) Enrichment: ToC 생성과 작성일 추론

ToC가 없는 원본 문서

🏠 금융안정보고서 (원본 문서)

원본 텍스트 (ToC 없음)

금융시스템 상황
금융기관 ...
은행 ...
금융시장 ...
...
금융안정 현안 분석
최근 국내외 금융여건 변화와...

LLM 생성 ToC (계층적 구조화)

제 I 장. 금융시스템 상황 1
I-1. 금융기관 2
가. 은행 2
I-2. 금융시장 18
제 II 장. 금융안정 현안 분석 35

// LLM 프롬프트 예시

금융안정보고서 전체 텍스트를 분석하여 계층적 목차(ToC)를 생성하세요.
각 챕터와 섹션의 페이지 번호를 포함하고 JSON 형식으로 반환하세요.
한국은행 문서 형식에 맞게 로마자 챕터 번호를 사용하세요.

출력 형식: 계층적 구조화 목차 JSON (id, title, page)

작성일이 없는 원본 문서

📅 금융안정보고서 (작성일 불명확)

원본 텍스트 (명시적 날짜 없음)

본 보고서는 11월 말 기준 가계부채 관련 최근 동향을 바탕으로, 금월 21일 금융통화위원회에서 논의된 주요 금융안정 리스크 요인을 분석하였습니다.
물가상승률 둔화세가 이어지는 가운데...

LLM 추론 결과

추론된 작성일: 2023-12-21
근거: "11월 말 기준", "금월 21일"
신뢰도: 0.92

// LLM 프롬프트 예시

금융안정보고서 전체 텍스트에서 문서의 작성일 또는 발행일을 추론하세요.
명시적 날짜가 없어도 상대적 표현(이번 분기, 전월, 금월 등)을 분석하세요.
날짜, 근거, 신뢰도 점수를 함께 반환하세요.

활용: 시간 기준 검색, 시계열 분석, 최신 문서 우선순위

LLM 기반 Enrichment 프로세스

문서 원문 입력 (구조 미정의)
유형별 LLM 프롬프트 생성
JSON 형태로 메타데이터 변환

문서 유형 자동 분류
정보 추출 및 구조화 추론
신뢰도 점수 계산

ToC 생성 - LLM 프롬프트 기반

문서에 목차가 없어도 자동 생성
계층적 구조 자동 파악
문서 유형별 최적 프롬프트 사용

문서 전체 텍스트 기반 추론
페이지 번호 추론 및 매핑
정규식 없이 고도 패턴 인식

작성일 추론 - LLM 프롬프트 기반

명시적 날짜 없는 문서도 처리
문서 내용 기반 시점 추론
다양한 날짜 형식 통일화

상대적 시간 표현 해석
신뢰도 점수와 근거 제공
시간대별 검색 지원

활용 방식

Section Header 기반 청킹 정확도 향상
시간순 정렬 검색 결과
스니펫에 섹션 제목 포함

섹션 기반 내비게이션 제공
기간 필터 검색 기능

LLM 프롬프트 기반 생성 장점

- 정규식 한계 극복: 복잡한 패턴도 문맥 이해로 정확히 추출
- 멀티모달 확장(VLM): 그림/차트 등 비텍스트 정보에 대한 텍스트 정보 생성
- 없는 정보 생성: 원본에 없는 ToC/작성일/이미지 설명도 자동 추론
- 문서 유형 자동 인식: 금융안정보고서, 통화정책보고서 등 구분
- 문맥 기반 신뢰도 평가: 불확실한 추론에 대한 신뢰도 제공

[세부 플로우] 2. 전처리

b. (지능형 전처리기) Enrichment: 이미지 설명 생성

이미지 설명이 없는 원본 문서

 프린터기 사용 메뉴얼 (원본 문서)

원본 이미지 (설명 없음)



LLM 생성 이미지 설명

> 이미지는 복합기 측면에 부착된 카드 리더기를 보여줍니다. 빨간 화살표와 숫자 1로 강조된 이 리더기는 직원증 인증을 위해 사용되며, 'CARD'와 'Sindeoh' 로고가 표시되어 있습니다. 이는 앞 문맥에서 언급된 직원증 인증 절차의 핵심 장비로, 인증 후 복사 화면이 변경되는지 확인하는 단계와 직접적으로 연결됩니다.

// LLM 프롬프트 예시
문서의 일부 이미지를 설명해줘. 아래 문맥을 참고해서 핵심 정보를 2~4문장으로 간결하게 작성해줘.

[앞 문맥] {{before_context}}

[캡션] {{caption}}

[뒤 문맥] {{after_context}}

요구사항:

1. 추측은 최소화하고 이미지에서 확인 가능한 사실 중심으로 작성
2. 문서 문맥과의 연결점을 포함
3. 한국어로 작성

출력 형식: PictureItem 주식 DescriptionAnnotation {kind, text, provenance}

LLM 기반 Enrichment 프로세스

문서 원문 입력 (구조 미정의)
유형별 LLM 프롬프트 생성
JSON 형태로 메타데이터 변환

문서 유형 자동 분류
정보 추출 및 구조화 추론
신뢰도 점수 계산

이미지 설명 생성 - LLM 프롬프트 기반

이미지에 대한 설명 자동 생성
같은 페이지 문맥 우선 수집
생성된 설명을 이미지에 주석으로 부착

주변 문맥(텍스트)을 프롬프트에 함께 입력
2~4문장의 간결한 설명 생성
이미지에서 확인 가능한 사실 중심 설명

활용 방식

이미지 설명이 청크에 포함 → 그림도 벡터 검색 대상에 포함됨
텍스트가 적은 이미지 위주 문서의 검색 커버리지 확보
그림/도표 관련 질의에 RAG 답변 근거 제공
검색 결과 스니펫에 이미지 설명 제공

LLM 프롬프트 기반 생성 장점

- 정규식 한계 극복: 복잡한 패턴도 문맥 이해로 정확히 추출
- 멀티모달 확장(VLM): 그림/차트 등 비텍스트 정보에 대한 텍스트 정보 생성
- 없는 정보 생성: 원문에 없는 ToC/작성일/이미지 설명도 자동 추론
- 문서 유형 자동 인식: 금융안정보고서, 통화정책보고서 등 구분
- 문맥 기반 신뢰도 평가: 불확실한 추론에 대한 신뢰도 제공

[세부 플로우] 2. 전처리

c. Chunking (1/2): 기본형 Token 길이 기반 그룹핑

원본 문서

금융안정보고서는 한국은행법 제96조 제1항에 의거하여 작성되며, 현행 규정상 연 2회 국회에 제출하게 되어 있습니다. 한국은행은 금융안정 상황을 종합적으로 분석·평가하고 있습니다.

지난해 이후 토큰 #510 글로벌 금융시장은 주요국 통화정책 향방에 대한 기대 변화 등에 영향받으며 변동성이 확대되었습니다. 물가상승 압력 지속 토큰 #512 등으로 중앙은행들의 정책금리 인상 기조가 강화되었습니다.

국내 금융시장에서는 토큰 #1020 시장금리가 주요국 통화정책 긴축 강화로 상승하였습니다. 원/달러 환율은 미 연준의 고강도 통화정책 긴축 토큰 #1024 등으로 원화 가치가 절하되었습니다.

Chunk 1 (토큰 #1-512)

금융안정보고서는 한국은행법 제96조 제1항에 의거하여 작성되며, 현행 규정상 연 2회 국회에 제출하게 되어 있습니다. 한국은행은 금융안정 상황을 종합적으로 분석·평가하고 있습니다.

지난해 이후 토큰 #510 글로벌 금융시장은 주요국 통화정책 향방에 대한 기대 변화 등에 영향받으며 변동성이 확대되었습니다. 물가상승 압력 지속 토큰 #512

100 토큰 오버랩 영역 ↓

Chunk 2 (토큰 #413-924)

토큰 #413 글로벌 금융시장은 주요국 통화정책 향방에 대한 기대 변화 등에 영향받으며 변동성이 확대되었습니다. 물가상승 압력 지속 등으로 중앙은행들의 정책금리 인상 기조가 강화되었습니다.

국내 금융시장에서는 토큰 #924

100 토큰 오버랩 영역 ↓

Chunk 3 (토큰 #825-1024+)

토큰 #825 국내 금융시장에서는 시장금리가 주요국 통화정책 긴축 강화로 상승하였습니다. 원/달러 환율은 미 연준의 고강도 통화정책 긴축 토큰 #1024 등으로 원화 가치가 절하되었습니다.

기본형 Token 길이 기반 청킹

기본 청킹 방식

- 텍스트를 고정 길이(일반적으로 512 토큰)로 분할
- 문맥 유지를 위해 100 토큰 중첩 영역 설정
- 텍스트 길이만 고려하며 내용의 의미는 무시

장점

- 간단한 구현 (토큰 수 기반 슬라이딩 윈도우)
- 벡터 크기 일관성 (고정 입력 길이 모델 호환)
- 실행 속도 빠름 (문서 구조 파싱 불필요)

단점

- 문장/문단/섹션 중간 분할로 의미 단절
- 문서 구조 정보 손실 (섹션/제목 관계 유실)
- 검색 정확도 저하 (중요 문맥 분리 가능성)

사용 사례

- 실시간 처리가 필요한 경우
- 빠른 처리 우선 파이프라인 (첨부용)
- 단순한 텍스트 문서 처리

[세부 플로우] 2. 전처리

c. Chunking (2/2): 지능형 Head 기반 그룹핑

I. 금융시장 동향

금융시장은 미 연준의 정책금리 인상 기대 변화 등에 영향받아...

1. 국내 금융시장

국내 금리는 주요국 금리 움직임에 연동되는 모습을 보였으며...

(1) 주식시장

주가는 경기 회복세가 기대에 미치지 못하면서 하락하였으나...

청크 #1

I. 금융시장 동향 > 1. 국내 금융시장 > (1) 주식시장

(1) 주식시장 주가는 경기 회복세가 기대에 미치지 못하면서 하락하였으나, 5월 들어서는 경기민감주를 중심으로 상승세로 전환하였다. KOSPI는 연초 이후 계속된 상승세가 4월 중 일부 조정되었으나, 5월 중 다시 상승하여 역대 최고치를 경신하였다.

2. 글로벌 금융시장

글로벌 주가는 주요국 경기지표 개선, 경기회복 기대감 등으로...

청크 #2

I. 금융시장 동향 > 2. 글로벌 금융시장

2. 글로벌 금융시장 글로벌 주가는 주요국 경기지표 개선, 경기회복 기대감 등으로 상승하였다. 선진국 국채금리는 물가상승 압력 등으로 상승하였다가 최근 통화정책 기조 전환 기대 등으로 하락하였다.

청킹 알고리즘 원리

기본 원칙

계층구조 활용: 보고서의 목차 체계(I → 1 → (1))를 기준으로 의미 단위 청크 생성
표/그림 독립처리: 표와 그림은 독립 청크로 분리하여 별도 처리
불필요 요소 제외: 페이지 번호, 헤더/푸터 등 검색에 불필요한 요소 제외

예외 규칙

작은 섹션 병합: 본문 길이가 짧은 섹션은 인접 본문과 병합(Chunk and Pad)
페이지 경계 무시: 섹션 내용이 페이지를 넘어가더라도 동일 섹션으로 포함

Header Field 구성

구조와 역할: 각 chunk의 첫 줄에 위치하며, 해당 chunk에 포함된 모든 계층의 section header를 한 줄로 기록
형식: "상위 섹션 > 하위 섹션 > 현재 섹션"

I. 금융시장 동향 > 1. 국내 금융시장 > (1) 주식시장

목적: chunk의 출처와 문서 내 위치를 명확히 표기하는 map 역할

벡터 검색 강화: semantic vector 계산 시 문맥 정보가 골고루 반영되도록 보장

활용 방식: 검색 결과의 정확한 출처 추적 및 관련 섹션 참조 가능

[세부 플로우] 3. Embedding



대상 필드: 본문 텍스트, 표 데이터(정규화), 메타데이터

적용 대상: 기본적으로 적재용 파이프라인에만 적용됨 (첨부용은 단순화된 처리)

- 📄 표 데이터: Markdown 형태로 변환하여 청크 내 텍스트에 포함
- 🖼️ 그림 데이터: BBox로 인식된 이미지 cropping 후 별도 저장, 메타데이터에 경로 저장 (적재용만)
- 🔗 임베딩 처리: 다국어 임베딩 모델로 텍스트 필드 벡터화
- 🔍 품질 개선: 금융 용어 도메인 튜닝, 토큰 오프셋 유지 (스팬 하이라이트)

임베딩 출력 형식

```
{
  "doc_id": "BOK_FR_2023_01",
  "chunk_id": "doc_123_chunk_7",
  "text": "금융안정보고서에 따르면...",
  "text_vector": [0.123, 0.456, ... -0.789]
}
```

[세부 플로우] 4. Vector DB 하이브리드 검색



하이브리드 검색 특징

Dense(의미) + Keyword(단어) 이중 인덱싱으로 검색 품질 극대화
 α 가중치 조정으로 검색 전략 최적화 ($\alpha=0.7$ 기본값)
복잡한 금융용어와 관용적 표현에도 강건한 검색 결과
한국어 특화 벡터 모델 적용 (다국어 지원)

성능 및 확장성

P95 응답시간 200ms 이하 (1GB 메모리 기준)
1억 문서 이상 확장 가능한 분산 벡터 DB 적용

검색 고도화 기능

메타데이터 필터링 (문서 유형, 작성일, 부서 등)
재순위화(Reranking) 적용 - 검색 정밀도 추가 향상
사용자 피드백 기반 검색 결과 품질 자동 개선

[전체 요약]



이원화 파이프라인

적재용(품질 중심)과 첨부용(속도 중심) 이중 구조로 사용 목적에 최적화



지능형 전처리

4가지 AI 모델로 문서 구조 자동 분석 및 이해 (OCR, Layout, Table, Reading Order)



의미 기반 Chunking

Section 헤더 기반 계층적 청킹으로 문맥 유지 최적화



하이브리드 검색

Dense + Keyword 인덱스 가중 결합으로 정확도와 재현율 균형

핵심 기술 요약



지능형 문서 분석

OCR·Layout·Table·Reading Order 4단계 AI 파이프라인
11가지 BBox 분류로 문서 구조 자동 해석 (Title, Text, Table 등)



문서 구조화 기술

LLM 기반 ToC 생성 및 작성일 추론 (없는 정보도 생성)
헤더 기반 계층적 청킹으로 의미 단위 분할



검색 최적화

$\alpha \cdot \text{dense} + (1-\alpha) \cdot \text{keyword}$ 가중 결합 방식
메타필드 필터링으로 다차원 검색 지원 (날짜, 출처 등)



성능 및 확장성

적재용·첨부용 이원화로 반응성과 품질 모두 확보
규정문서 등 특수 문서 전용 처리기 적용



핵심 데이터 포맷

doc_id chunk_id text text_vector metadata

2. 전처리 별 문서 변환/전처리 옵션 도해

- 첨부 전처리기
 - 문서 확장자별 개별 전처리 Flow
- 적재 전처리기
 - PDF 기반 지능형 전처리 Flow
 - 선택 사항: Office File → PDF 변환 방식 선택
 - [별첨] 현재 전처리별 HWP 처리 형태
 - 적재 문서 실무 적용 가이드 (PDF 변환)

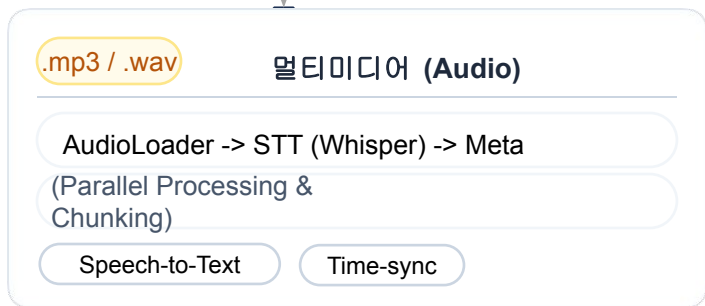
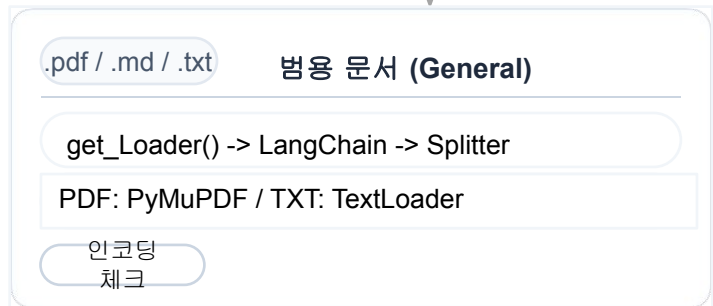
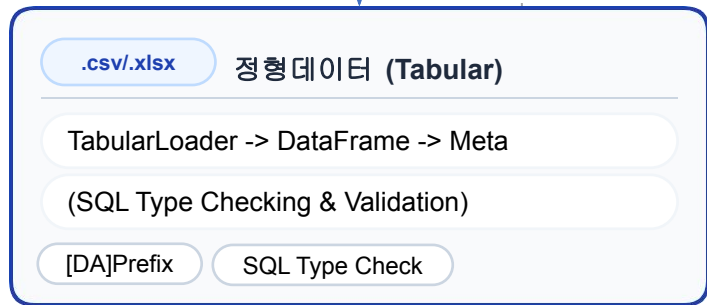
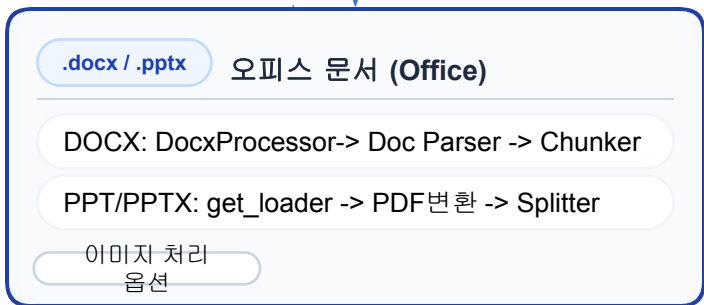
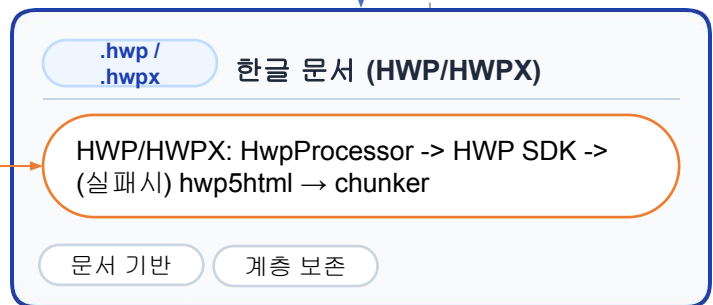
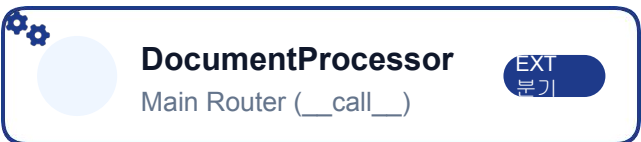
[첨부 전처리기] - 문서 확장자별 개별 전처리 Flow

속도 중심: 다양한 포맷의 텍스트 즉시 추출

확장자(Extension) 기반 실시간 라우팅

경량화된 리소스 사용

※ hwp5html은 오픈소스로서 HTML 구조로 변환한 뒤 PDF를 거쳐 텍스트를 추출하지만, 오류를 자주 동반. Genos가 공식적으로 지원가능한 솔루션이 아님. 현재는 상용솔루션(HWP SDK)으로 이부분을 강화함.



[적재 전처리기] - PDF 기반 지능형 전처리 Flow

품질 중심: AI 기반 Layout 분석 및 고품질 데이터 적재

PDF 단일 경로 파이프라인

RAG 기반 최적화

1 분석 (Analysis)

01 PDF 기반 Layout Parsing

딥러닝 비전 모델로 11종 요소 식별 및 구조 분석

Layout Detection

Table Former

02 품질 검사

텍스트 추출 품질 및 GLYPH(깨짐) 자동 감지

check_glyphs

Integrity Check

03 선택적 OCR

문제가 감지된 테이블 셀 단위로 제한적 OCR 수행

Paddle OCR

Cell-Level

2 보강 (Enrichment)

04 LLM Enrichment

LLM을 활용한 계층적 목차(TOC) 생성 및 메타데이터 추출

LLM / VLM

ToC 생성

05 이미지 참조 설정

문서 내 이미지 추출 및 경로 참조 연결

Extract Image

06 빈 문서 처리

텍스트 부재 시 더미 데이터 삽입으로 프로세스 중단 방지

Fallback

Dummy Insert

3 구조화 (Structuring)

07 Chunking

Bucket Chunker

토큰 단위로 Bucket 을 조정 (0~8192)

- 순수 섹션 단위로 문서 분할 (Bucket : 0)

- Size 허용치까지 item군 별로 분할 (0<Bucket <8192)

Sectioning

100% Structure

08 부록 자동 연결

본문 내 '별지/별표' 참조를 감지하여 파일 자동 매칭

Appendix Matching

정규식

09 Meta 조립

최종 벡터 메타데이터 생성 (HEADER, bbox, appendix 포함)

Rich Metadata

Key Features



AI Layout Analysis

제목, 본문, 표, 그림 등
11종 문서 요소 정밀
식별



Table 인식

병합 셀, 다중 헤더 표를
완벽한 마크다운으로
복원



Smart OCR

인코딩이 깨진(GLYPH)
영역만 선별적으로
복구



LLM Enrichment

문서 구조를 이해하여
목차 및 메타데이터
생성

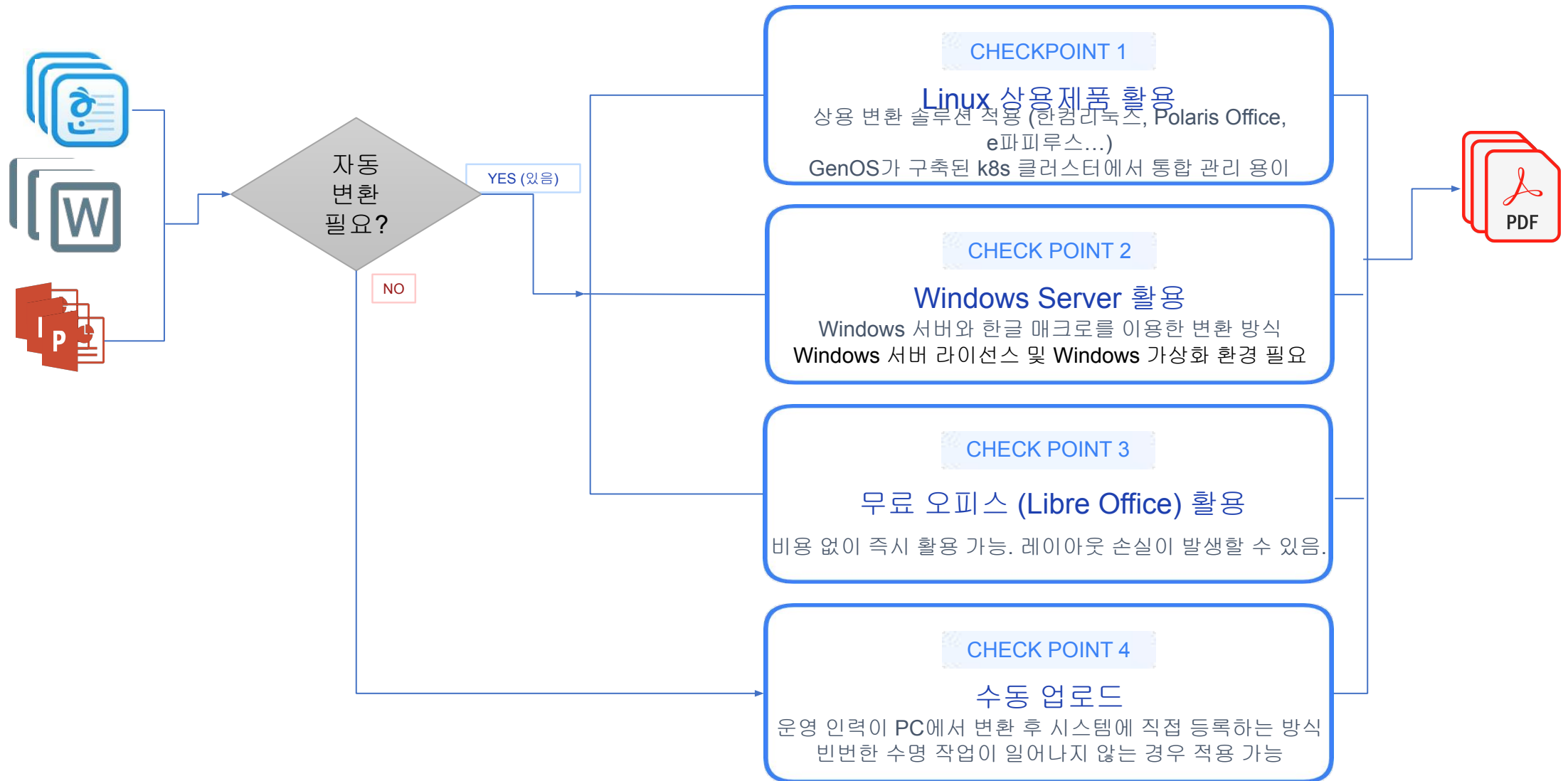


Appendix Linking

본문과 부록 파일을
자동으로 연결하여
검색

[적재 전처리] - PDF 기반 지능형 전처리 Flow

- [선택 사항] Office File → PDF 변환 방식 선택



[적재 전처리] - PDF 기반 지능형 전처리 Flow

- [별첨] 현재 전처리별 HWP 처리 하는 형태



첨부용 전처리기

Ad-hoc (실시간 처리)



Process Logic

CPU를 이용하여 문서의 text 정보 위주로 빠른 시간에 추출.
별도의 viewer 없이 채팅창에서 첨부하고 사용자가 입력한 채팅결과로 처리.



HWP → PDF (rhwp 또는 LibreOffice)

현재 방식: 텍스트 추출 전용

비용 없이 즉시 변환 가능하나 품질 한계 존재

단순 텍스트 가능

표 데이터 깨짐

레이아웃 미보장

</> HWPX 파서 (내장)

엔진 내부 파싱 기능 활용

HWPX 포맷에 대해서는 구조적 파싱 지원

HWP → HWPX 변환
미제공

End User HWP 첨부 불가

HWP → 상용 컨버터(HWP SDK) 사용 → 컨버팅 된 결과
활용

한국은행 레퍼런스 방식

CPU 기반 빠르고 정확한 표/그림 추출 가능

자유소프트 SDK

한컴/자유소프트



적재용 전처리기

Admin (사전 적재)



Process Logic

관리자 문서 적재 → HWP를 PDF로 변환 → Layout 예측 및 지능형 추출.
Viewer를 통해 관리자가 추출 결과를 검증.



1. Linux 기반 (자동/서버)

한글오피스 리눅스 / 3rd Party

CLI 제공, HWP → HWPX/PDF 변환

Polaris Office, ePapyrus 등 변환기

서버 기반 자동화

대량 처리 적합



2. Windows 기반 (자동/서버)

Windows Server / VMWare 가상화

한글 매크로 프로그램 활용

가상화 환경 구축 필수

OS 라이선스 비용

인프라 구축 필요



3. 수동 업로드

운영 인력이 PC 변환 후 등록

주기적인 수동 작업 필요

초기 도입 시 적합

운영 공수 증가

라이선스 불필요

[적재 전처리기]

- 적재 문서 실무 적용 가이드 (PDF 변환)

1

Linux 서버 환경용 상용 제품 가능한가?

안정적 서버 운영 자동화 및 안정적 HWP -> PDF 변환 품질
요구
상용제품 예. 한컴 Office 리눅스, Polaris Office, ePopyrus

YES



Option 1 (Linux 상용)

2

Windows 환경(VM) 구축이 가능한가?

기존 Windows Server 라이선스 활용 또는 VM 운영 가능

YES



Option 2 (Win 자동화)

3

운영 인력 투입이 가능한가?

자동화 시스템 구축 전 초기 단계, 비용 최소화

YES



Option 3 (수동 업로드)

4

HWP/HWPX 파일인가?

무료 솔루션임에도 LibreOffice 보다 우수한 품질의 변환 제공
(HWP/HWPX 한정)

YES



Option 4 (rhwp)

5

마지막 선택: 레이아웃 타협 가능한가?

단순 텍스트 추출 목적, 표/이미지 깨짐 감수

YES



Option 5 (LibreOffice)

💡 사용 목적별 추천



사용자 원본 열람용

화면 깨짐 없는 완벽한 변환 품질 필수

👉 Option 1 (Linux 상용) 권장



임시 / 소규모 운영

초기 비용 0원으로 시작

👉 Option 3 (수동 업로드) 권장



단순 첨부 텍스트 추출

사용자가 첨부한 pdf 에서 단순. 텍스트
추출

👉 Option 4 (내장 rhwp 활용) 또는
Option 5 (내장 office 활용) 권장

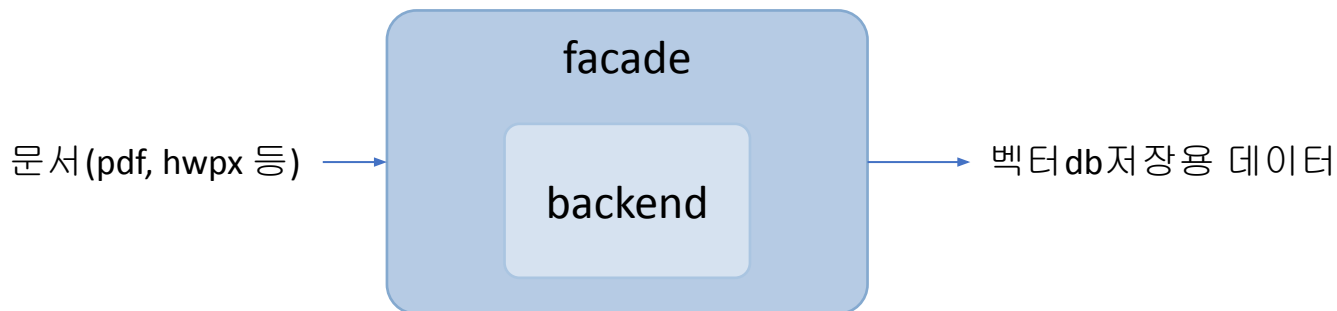
3. 전처리 프로젝트 소스 사용 방법

- 개요
 - Github 주소
 - 전처리기 구성
- 환경 구성
 - 프로젝트 환경 구성
 - 테스트 위치
 - 테스트 코드
- 테스트 설정
 - Facade 코드
 - Intelligent_processor.py 메인 함수
 - YAML 파일을 통한 Facade 내부 값 설정
 - Enrichment 프롬프트 수정 안내

전처리기 프로젝트 소스 사용 방법

- 개요

- Github 주소
 - https://github.com/genonai/doc_parser/tree/develop
 - Develop 브랜치가 최신 버전
- 전처리기 구성
 - Backend:
 - 문서파싱 엔진, IBM docling기반
 - Facade
 - 문서파싱 이후 청킹, 메타데이터 추출 등을 수행
 - Genos 에서 코드 수정가능
 - 사이트에서는 상황에 따라 facade를 구성해서 활용
 - 입력파일 포맷 변경
 - 메타데이터 변경 등



프로젝트 구조

```
# 최상단은 docling 구조를 그대로 가져감
# doc-parser 관련은 genon 디렉터리에 작성

.
├── build-script # 빌드 위한 스크립트 파일 및 컨피그 파일
├── docling
├── docs
├── genon
│   ├── preprocessor # genos에서 실행 될 전처리기 이미지 및 facade 관련
│   │   ├── configs # gunicorn, supervisor 설정
│   │   ├── docker # 도커파일 위치
│   │   ├── env # 개발 시 설정 파일들
│   │   └── facade # facade 코드
│   │       ├── evaluation
│   │       │   ├── test_files
│   │       │   │   ├── annotated
│   │       │   │   ├── pdf
│   │       │   │   └── result
│   │       ├── gitbook_doc
│   │       │   └── images
│   │       ├── legacy
│   │       ├── legal_parser
│   │       │   ├── api
│   │       │   ├── commons
│   │       │   ├── parsers
│   │       │   ├── schemas
│   │       │   └── services
│   │   ├── scripts # 도커 이미지 push 및 디비 등록 관련 스크립트 위치
│   │   ├── resources # 폰트 및 기타 리소스 파일들
│   │   ├── sample_files
│   │   ├── src # 전처리기 API 소스
│   │   │   └── common
│   │   └── tests # genos doc-parser 테스트 소스
│   │       ├── regression
│   │       │   └── baselines
│   │       ├── smoke
│   │       └── unit
│   ├── serving # ocr 및
│   │   └── paddle
│   │       ├── config # ocr, vl paddlex 실행 파일
│   │       ├── docker
│   │       ├── etc
│   │       ├── k8s-manifest
│   │       └── resources
│   └── tools
│       ├── genos_tools
│       └── commands
└── tests # docling 리포 test 관련 코드 작성 x
```

전처리기 프로젝트 소스 사용 방법

- 환경 구성

- 프로젝트 환경 구성

```
git clone https://github.com/genonai/doc_parser.git
cd doc_parser
git checkout develop
cd ./genon/preprocessor/
uv sync
source ./venv/bin/activate
```

- 테스트 위치

```
cd ./facade
python test.py
result.json 파일에 결과 저장됨
```

**** genon vpn 설정필요 ****

```
[
  {
    "text": "HEADER: A2, English Practice Test, SECTION I: Grammar (Use of English), SE0",
    "n_char": 3650,
    "n_word": 610,
    "n_line": 110,
    "e_page": 4,
    "i_page": 1,
    "i_chunk_on_page": 0,
    "n_chunk_of_page": 1,
    "i_chunk_on_doc": 0,
    "n_chunk_of_doc": 4,
    "n_page": 11,
    "reg_date": "2025-11-20T10:13:46Z",
    "chunk_bboxes": "[{\n\"page\": 1, \n\"bbox\": {\n\"l\": 0.12116814906301063, \n\"t\": 0.9630",
    "media_files": "[{\n\"name\": \n\"image_000000_28a02bee8565751499d6a497834b73d0b02c97163",
    "title": "",
    "created_date": 0
  },
  {
    "text": "HEADER: WELCOME TO CYBERSPACE!, Reading Task 2 Read the article and mark th",
    "n_char": 4679,
    "n_word": 788,
    "n_line": 64,
    "e_page": 7,
    "i_page": 4,
    "i_chunk_on_page": 0,
    "n_chunk_of_page": 1,
    "i_chunk_on_doc": 1,
    "n_chunk_of_doc": 4,
    "n_page": 11,
    "reg_date": "2025-11-20T10:13:46Z",
    "chunk_bboxes": "[{\n\"page\": 4, \n\"bbox\": {\n\"l\": 0.11920098039215685, \n\"t\": 0.5930",
    "media_files": "[{\n\"name\": \n\"image_000005_d629f73e6777afb3f2e4278aa10745faa016e7d49",
    "title": "",
    "created_date": 0
  },
  {
    "text": "HEADER: Reading Task 4 Read the letter, then answer the questions below., F",
    "n_char": 4476,
    "n_word": 771,
```

<result.json 결과 예시>

전처리기 프로젝트 소스 사용 방법

- 환경 구성

- 테스트 코드
 - [test.py](#)

```
# 테스트할 전처리기 임포트
# from attachment_processor import DocumentProcessor # 첨부용
# from convert_processor import DocumentProcessor # 변환형
from intelligent_processor import DocumentProcessor # 지능형
```

```
# 파일 경로
file_path = "../sample_files/pdf_sample.pdf" □ 테스트할 문서 파일 경로 지정
```

```
import os
from fastapi import Request
import logging
import asyncio
import json
import time

import sys
sys.path.insert(0, "../..") # 현재 doc_parser의 docling 폴더 참조

# 테스트할 전처리기 임포트
# from attachment_processor import DocumentProcessor # 첨부용
# from convert_processor import DocumentProcessor # 변환형
from intelligent_processor import DocumentProcessor # 지능형

# 파일 경로
file_path = "../sample_files/pdf_sample.pdf"

# 파일 존재 여부 확인
if not os.path.exists(file_path):
    print(f"Sample file not found: {file_path}")
    print("Please add a file to the sample_files folder.")
    exit(1)

# DocumentProcessor 인스턴스 생성
doc_processor = DocumentProcessor()

# FastAPI 요청 예제
mock_request = Request(scope={"type": "http"})

# 비동기 메서드 실행
async def process_document():
    print(file_path)
    kwargs = {}
    kwargs['org_filename'] = os.path.basename(file_path)
    vectors = await doc_processor(mock_request, file_path, **kwargs)
    return vectors

begin = time.time()
# 메인 루프 실행
result = asyncio.run(process_document())

result_list_as_dict = [item.model_dump() for item in result]

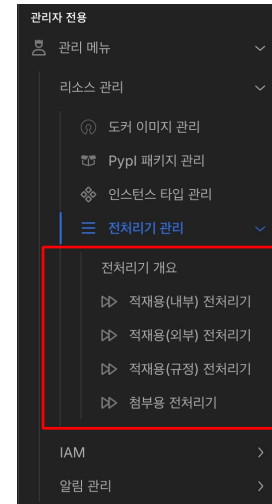
# 최종적으로 이 리스트를 JSON으로 저장
with open("result.json", "w", encoding="utf-8") as f:
    json.dump(result_list_as_dict, f, ensure_ascii=False, indent=4)

end = time.time()
print(f"Processing time: {end - begin:.2f} seconds")
```

전처리기 프로젝트 소스 사용 방법

- 테스트 설정

- Facade 코드
 - [메뉴얼 설명](#)
- [intelligent_processor.py – 메인 함수](#)
 - facade 코드에 대한 자세한 설명은 GenOS Docs 참고



```
... async def __call__(self, request: Request, file_path: str, **kwargs: dict):
...     # kwargs['save_images'] = True ... # 이미지 처리
...     # kwargs['include_wmf'] = True ... # wmf 처리
...     document: DoclingDocument = self.load_documents(file_path, **kwargs)
...
...     if not check_document(document, self.enrichment_options) or self.check_glyphs(document):
...         # OCR이 필요하다고 판단되면 OCR 수행
...         document: DoclingDocument = self.load_documents_with_docling_ocr(file_path, **kwargs)
...
...     # 글리프 깨진 텍스트가 있는 테이블에 대해서만 OCR 수행 (청크토큰 8k이상 발생 방지)
...     document: DoclingDocument = self.ocr_all_table_cells(document, file_path)
...
...     output_path, output_file = os.path.split(file_path)
...     filename, _ = os.path.splitext(output_file)
...     artifacts_dir = Path(f"{output_path}/{filename}")
...     if artifacts_dir.is_absolute():
...         reference_path = None
...     else:
...         reference_path = artifacts_dir.parent
...
...     document = document._with_pictures_refs(image_dir=artifacts_dir, reference_path=reference_path)
...
...     document = self.enrichment(document, **kwargs)
```

docling 기반 문서파싱

ocr 필요성 판단
ocr 필요하면 다시 docling 문서파싱.
문서파싱과정에서 ocr 수행

표에 대해서 ocr 수행 (글리프 깨진 경우만)

파싱 데이터(document) 에서
이미지를 별도 파일로 저장

enrichment 수행
ToC 추출, 작성일 추출

전처리기 프로젝트 소스 사용 방법

- 테스트 설정

- Facade 코드
 - [intelligent_processor.py](#) – 메인 함수

```
.....has_text_items = False
.....for item, _ in document.iterate_items():
.....    if (isinstance(item, (TextItem, ListItem, CodeItem, SectionHeaderItem)) and item.text and item.text.st
.....        has_text_items = True
.....        break

.....if has_text_items:
.....    # Extract Chunk from DoclingDocument
.....    chunks: List[DocChunk] = self.split_documents(document, **kwargs)

.....else: ""
.....    # await assert_cancelled(request)

.....vectors = []
.....if len(chunks) >= 1:
.....    vectors: list[dict] = await self.compose_vectors(document, chunks, file_path, request, **kwargs)
.....else:
.....    raise GenosServiceException(1, f"chunk length is 0")

.....""" ""

.....return vectors
```

파싱데이터에서 텍스트 존재하는지 검사

청킹

벡터DB저장형태로 변환 및
메타데이터 추가

벡터DB저장형태 반환

전처리기 프로젝트 소스 사용 방법

- 테스트 설정

- Facade 코드
 - [intelligent_processor.py – YAML을 로드하는 부분](#)

‘resource_dev/intelligent_processor_config.yaml’을 우선으로 로드.
→ 미존재시, ‘resource/intelligent_processor_config.yaml’을 로드.

YAML에서 설정된 값을 기반으로,
OCR, Layout, PDF 파이프라인, Enrichment 값이 설정됨

```
def _resolve_default_intelligent_config_path() -> str:
    base_dir = Path( file ).resolve().parent
    local_config = (base_dir / "../resource_dev/intelligent_processor_config.yaml").resolve()
    default_config = (base_dir / "../resource/intelligent_processor_config.yaml").resolve()

    if local_config.exists():
        return str(local_config)
    return str(default_config)
```

```
class DocumentProcessor:

    def __init__(self, config_path: str | None = None):
        """
        initialize Document Converter (config 기반)

        config_path 가 None 이면 resource_dev/intelligent_processor_config.yaml
        (없으면 resource/intelligent_processor_config.yaml) 을 사용한다.
        GenOS 는 DocumentProcessor() 무인자로 호출하므로 기본 경로 resolve 필수.
        """
        if config_path is None:
            config_path = _resolve_default_intelligent_config_path()

        cfg = _load_config(config_path)
        self._config_dir = Path(config_path).resolve().parent

        defaults_cfg = _as_dict(cfg.get("defaults"))
        log_level = _parse_optional_int(defaults_cfg.get("log_level"), "defaults.log_level")
        if log_level is None:
            log_level = 4
        self._log_level = log_level

        ocr_cfg = _as_dict(cfg.get("ocr"))
        layout_cfg = _as_dict(cfg.get("layout"))
        pdf_cfg = _as_dict(cfg.get("pdf_pipeline"))
        ec = EnrichmentConfig.from_raw(cfg.get("enrichment"), self._config_dir, parent_cfg=cfg)

        # OCR 엔드포인트는 ocr.paddle.ocr_endpoint 가 정식 위치.
        # 구버전 호환: ocr.ocr_endpoint(상위) / 최상위 ocr_endpoint 도 폴백으로 인식.
        paddle_cfg = _as_dict(ocr_cfg.get("paddle"))
```

전처리기 프로젝트 소스 사용 방법

- 테스트 설정

- YAML 파일을 통한 Facade 내부 값 설정
 - [intelligent_processor.yaml - Docling 주요 설정 옵션](#)
 - [YAML 사용 메뉴얼 링크](#)

PDF 파이프라인 옵션

```
# PDF 파이프라인 (docling PDF 파싱 성능/품질 노브)
pdf_pipeline:
  num_threads: 8                # accelerator 스레드 수
  device: "auto"                # "auto"(default) | "cpu" | "cuda" | "mps"
  images_scale: 2               # 페이지/그림 이미지 렌더 배율
  generate_page_images: true
  generate_picture_images: true  # false 면 이미지 설명 enrichment 비활성화됨
  table_structure_mode: "accurate" # "accurate"(default) | "fast"
```

Layout 모델 옵션

```
layout:
  layout_model_type: "genos_layout" # "genos_layout"(default) or "docling_layout"
  genos_layout:
    # <LAYOUT_SERVING_ID>: Genos에 등록된 layout 모델서빙 ID로 변경 필요
    # api_key는 k8s 내부 통신 기반 모델 호출 시 불필요.
    endpoint: "http://llmops-gateway-api-service:8080/rep/serving/<LAYOUT_SERVING_ID>/v1/chat/completions"
    api_key: ""
    page_batch_size: 32
    max_completion_tokens: 16384
```

Layout 서빙 주소

OCR 모델 옵션

```
ocr:
  ocr_mode: "auto" # "auto"(default), "force", "disable"
  # OCR 엔진 선택. "paddle"(default) | "upstage"
  engine: "paddle"
  # 글리프 깨진 테이블 셀 재OCR 시 HTTP timeout(초)
  table_cell_ocr_timeout: 60

  paddle:
    # engine: "paddle" 일 때만 사용
    # <OCR_ENDPOINT>: PaddleOCR 서버 주소로 변경 필요
    ocr_endpoint: "http://<OCR_ENDPOINT>/ocr"
    text_score: 0.3

  # 글리프 깨짐 기반 auto-OCR 재트리거 임계값
  glyph_detection:
    table_cell_threshold: 1 # 셀 GLYPH 토큰 N개 이상이면 재OCR
    document_threshold: 10 # 문서 GLYPH 토큰 N개 초과면 OCR 경로 재시도

  # engine: "upstage" 일 때만 사용
  upstage:
    api_endpoint: "https://api.upstage.ai/v1/document-digitization"
    # api_key 가 비어있으면 UPSTAGE_API_KEY 환경변수에서 fallback
    api_key: ""
    model: "ocr"
    timeout: 60
    text_score: 0.5
```

'force' 설정시 강제 ocr 수행

ocr 엔진 선택

ocr 서빙 주소
(paddle 사용시)

ocr 서빙 주소
(upstage 사용시)

genos_layout → dotsoocr 모델
docling_layout → docling 내장
모델

전처리기 프로젝트 소스 사용 방법

- 테스트 설정

- YAML 파일을 통한 Facade 내부 값 설정
 - intelligent_processor.yaml - Enrichment 옵션 (일반문서)

Enrichment 옵션

1. Toc 설정 옵션

```
enrichment:
- toc:
  enable: true
  url: "http://llmops-gateway-api-service:8080/rep/serving/"
  api_key: ""
  model: "model"
  temperature: 0.0
  top_p: 0.00001
  seed: 33
  max_tokens: 10000
  precheck:
    enabled: true
    max_context_tokens: 128000
    completion_reserved_tokens: 12000
  system_prompt_file: prompt_toc_default_system.md
  user_prompt_file: prompt_toc_default_user.md
```

생성 샘플링 제어값.
(0에 가깝게 = 매번 같은 결과)

생성 최대 토큰.
목차가 길어서 크게 (10000)
입력이 모델 컨텍스트를 넘는지 사전 검사.

2. Metadata 설정 옵션 (작성일 등)

```
- metadata:
  enable: true
  url: "http://llmops-gateway-api-service:8080/rep/serving/"
  api_key: ""
  model: "model"
  max_tokens: 10000
  temperature: 0.0
  timeout: 3600
  pages: [1,4] # null = 첫 4페이지
  # output_fields: 프롬프트의 JSON 키와 일치해야 함
  output_fields: [created_date, authors]
  parser:
    type: json # json(default) | python
  system_prompt_file: prompt_metadata_default_system.md
  user_prompt_file: prompt_metadata_default_user.md
  precheck:
    enabled: true
    max_context_tokens: 128000
    completion_reserved_tokens: 12000
```

생성 샘플링 제어값.
(0에 가깝게 = 매번 같은 결과)
요청 시간제한 (초)

메타데이터 뽑을 페이지 범위
추출할 필드명.

응답 결과 파싱 방식

입력이 모델 컨텍스트를 넘는지 사전 검사.

3. Image_description 설정 옵션

```
- image_description:
  enable: false
  url: "http://llmops-gateway-api-service:8080/rep/serving/"
  api_key: ""
  model: "model"
  concurrency: 16 # 이미지 설명 요청 병렬 수
  before_items: 3
  after_items: 2
  max_context_chars: 1500
  prompt_template_file: prompt_image_description_default.md
```

이미지 설명 생성 요청 동시 처리 개수 (병렬)
이미지 앞/뒤에서 가져올 문맥 텍스트 개수 (앞3, 뒤2)
프롬프트에 넣을 문맥 글자 최대 수

(공통 인자 설명)

- > enable: 각 작업 수행 유무
- > url: llm 서버 주소 (셋이 동일)
- > api_key: 모델 접근 key, 내부서빙모델은 설정 불필요 (셋이 동일)
- > model: 모델지정, 내부서빙모델은 설정 불필요 (셋이 동일)
- > system_prompt_file: 사용자 지정 시스템 프롬프트 파일 (작업마다 다름)
- > user_prompt_file: 사용자 지정 요청 프롬프트 파일 (작업마다 다름)
- > image_description에서 prompt_template_file 하나로 존재

전처리기 프로젝트 소스 사용 방법

- 테스트 설정

- YAML 파일을 통한 Facade 내부 값 설정
 - [intelligent_processor.yaml](#) - url 설정 관련

모델 서빙 endpoint 설정 (내부 / 외부)

환경	URL 형태	api_key
GenOS 배포 컨테이너가 k8s 내부	<code>http://llmops-gateway-api-service:8080/rep/serving/<ID>/...</code>	공란 OK (내부 통신)
로컬 PC 외부에서 호출	<code>https://genos.genon.ai/api/gateway/rep/serving/<ID>/...</code>	필요 (외부라 인증)

<ID> 값은, GenOS에 등록된 자원의 ID를 입력.

모델 서빙 목록

서빙 생성 +		전체 서빙	
ID	서빙 유형	제목	상세 설명
784	VLM	교육용 모델 서빙	
782	VLM	테스트용 모델서빙	
781	LLM	[개발팀] api.genon.ai	

사내망 로컬 PC에서 사용시 예시

```
# -- enrichment --
# 각 항목은 {이름: {옵션}} 형식의 list.
# 비활성화 방법: ① 항목 삭제 ② 항목 주석 처리 ③ enable: false 추가 (설정 보존)
enrichment:
- toc:
  enable: true
  # 동작 보존: 기존 하드코딩 값(서빙 502)을 그대로 사용
  url: "https://genos.genon.ai/api/gateway/rep/serving/750/v1/chat/completions"
  api key: "26e6a516de0847f59a3b9db8e418976e"
  model: "model"
```

Genos에 배포시 예시

```
# -- enrichment --
# 각 항목은 {이름: {옵션}} 형식의 list.
# 비활성화 방법: ① 항목 삭제 ② 항목 주석 처리 ③ enable: false 추가 (설정 보존)
enrichment:
- toc:
  enable: true
  # 동작 보존: 기존 하드코딩 값(서빙 502)을 그대로 사용
  url: "https://llmops-gateway-api-service:8080/rep/serving/750/v1/chat/completions"
  api_key: ""
  model: "model"
```

Github에 명시된 [‘resource dev/intelligent_processor config.yaml’](#)은, 사내망 로컬 PC에서 동작하도록 설정된 상태
→ Genos 배포시에는, url 교체 및 api_key 제거 필요.

전처리기 프로젝트 소스 사용 방법

- 테스트 설정

- Enrichment 프롬프트 수정 안내 — (1) custom_fields 수정 vs (2) user_prompt 수정

프롬프트를 기호에 맞게 고치는 방법은 목적에 따라 둘로 나뉨
(기본적으로는 수정을 하지 않아도 동작. 기호에 따라 수정이 필요할 경우에만 참고)

① 새 정보를 더 뽑으려면 → custom_fields 추가

- metadata(작성일, 작성자) 외 새 필드 신설
예: 부서 / 문서번호 / 승인자 등
- user_prompt에 직접 새 필드 쓰지말 것(권장 X)
- 상세는 다음 슬라이드 참조

② 출력 방식을 바꾸려면 → user_prompt 수정

- 언어 / 길이 / 스타일 / 상세도 조정 (추출 항목은 그대로)
- 예시: image_description
기본 설정은 '한국어 2~4문장' → 영어로 / 1문장으로 / 더 상세히 등으로 수정 가능
- toc, metadata도 수정 가능하나, 출력 형식 부분은 수정하지 않아야 함(파싱 오류 발생).

⚠ user_prompt 수정 시 주의 사항(파싱 오류 유발 원인)

- {{raw_text}} 는 유지(이중 중괄호) · ToC 출력형식(<toc>·TITLE·십진번호)은 고정해야함.

전처리기 프로젝트 소스 사용 방법

- 테스트 설정

- Enrichment 프롬프트 수정 안내 — (1) custom_fields 추가 및 수정

custom_fields란? - metadata의 '사용자 정의' 버전 — 원하는 필드를 직접 정의해 LLM으로 추출

핵심

- 인자는 metadata와 동일
(url·model·pages·output_fields·parser·프롬프트파일 등)
- 복수 항목 등록 가능
- 기본 enable: false

등록 방식 2가지:

- ㉠ 인라인 — yaml에 직접 내용 추가
- ㉡ 외부 파일에서 가져오기 — config_file을 통해 가져오도록

예시:

- [resource dev/intelligent_processor config.yaml](#)

예시 (authors 추출 예시)

㉠ 인라인 — yaml에 직접 내용 추가

```
- custom_fields:
  enable: true
  {.....(url, model 등의 인자) .....}
  output_fields: [authors] # 뽑을 필드
  pages: [1]
  parser: { type: json }
  system_prompt_file: prompt\_custom\_fields\_authors\_system.md
  user_prompt_file: prompt\_custom\_fields\_authors\_user.md
```

㉡ 외부 파일에서 가져오기 — config_file을 통해 가져오도록

```
- custom_fields:
  enable: true
  config_file: custom\_field\_authors.yaml
```

- [resource dev/intelligent_processor config.yaml](#) 참고: [resource/intelligent_processor.yaml](#) 에는 custom_fields 정의가 없다. dev를 참고해야함.
- 추가된 field에 대해서, 해당 field를 추출하기 위한 시스템 프롬프트와 사용자 프롬프트 생성후, 이를 명시해야함
→ [prompt_custom_fields_authors_system.md](#)과 [prompt_custom_fields_authors_user.md](#)이 그 예시

전처리기 프로젝트 소스 사용 방법

- 테스트 설정

- Enrichment 프롬프트 수정 안내 — (2) user_prompt 수정
[‘preprocessor/resource/ **default.md’ - Enrichment의 Prompt 파일](#)

● 자유 수정 가능
● 수정 불가(시스템 프롬프트, 출력 형식에 영향 등의 사유)

prompt_image_description_default.md

(이미지 설명 생성 프롬프트)

문서의 일부 이미지를 설명해줘.
아래 문맥을 참고해서 핵심 정보를 2~4문장으로 간결하게 작성해줘.

[앞 문맥] {{before_context}}
[캡션] {{caption}}
[뒤 문맥] {{after_context}}

요구사항:

1. 추측은 최소화하고 이미지에서 확인 가능한 사실 중심으로 작성
2. 문서 문맥과의 연결점을 포함
3. 한국어로 작성

● 수정 예시

1. 길이(2~4문장 → 1문장 or 더 상세히)
2. 언어(한국어→영어)
3. 말투/강조점

● 수정 불가:

1. 제거시 VLM에게 전달되는 문맥 정보 누락.

prompt_metadata_default_system.md

prompt_metadata_default_user.md

(Metadata 추출 관련 시스템 프롬프트)

You are a professional document extraction assistant.
Your job is to carefully extract metadata from semi-structured or unstructured Korean financial documents.
Always follow the requested output format exactly.

● 수정 불가 - 시스템 프롬프트에 해당

prompt_toc_default_system.md

prompt_toc_default_user.md

(Toc 추출 관련 시스템 프롬프트)

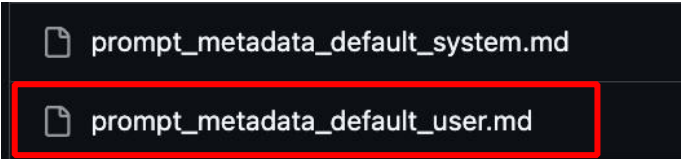
You are an expert at generating table of contents (목차) from Korean documents.
You specialize in regulatory documents, terms of service, contracts, and mixed-format documents that combine formal regulatory structures with general section headers.

● 수정 불가 - 시스템 프롬프트에 해당

전처리기 프로젝트 소스 사용 방법

- 테스트 설정

- Enrichment 프롬프트 수정 안내 — (2) user_prompt 수정
'preprocessor/resource/ **default.md' - Enrichment의 Prompt 파일



(Metadata 추출 관련 사용자 프롬프트)

다음은 한국어로 작성된 금융 보고서 문서의 일부입니다. 이 문서에서 아래 정보를 정확히 추출해주세요:

1. 최초 작성일 (날짜가 여러 개 있으면 '보고자료' 또는 '회의자료'에 가까운 날짜 사용)
2. 작성자 이름과 소속 또는 직책 (예: 채권시장팀 조용범 과장)
3. 작성자 전화번호 (내선번호 포함 시 함께 출력)

문서:
{{raw_text}}

출력 형식은 반드시 아래와 같이 맞춰주세요 (JSON 형식):

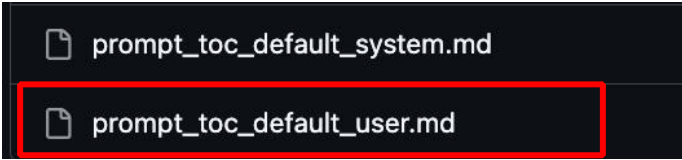
```
{
  "created_date": "YYYY-MM-DD",
  "authors": [ {"name": "조용범", "affiliation": "채권시장팀", "position": "과장", "phone": "4751"},
               {"name": "임인혁", "affiliation": "주식시장팀", "position": "과장", "phone": "4672"} ]
}
```

● 수정 예시

1. 작성일 판단 기준 수정
(예: 날짜가 여러개면 마지막 언급된 날짜)
2. 작성자 판단 기준 수정
(예: 여러명이면 대표 1인만 추출 or 전원 추출 등)

● 수정 불가:

1. {{raw_text}} 제거시 문서 정보 누락
2. 출력 형식에 영향을 줌.



(Toc 추출 관련 사용자 프롬프트)

Here is the Korean document you need to analyze:
{{raw_text}}

Your task is to extract and organize all structural elements from this document into a hierarchical table of contents. Korean documents often have mixed structures where some sections follow formal regulatory patterns (제x장/절/관/조) while others use general section numbering and headers.

Analysis Process

Before generating the final table of contents, work through the document systematically in <analysis> tags. It's OK for this section to be quite long. Follow these steps:

1. **Document Title Extraction:** Quote the main document title exactly as it appears at the beginning of the document.

.....(뒤 슬라이드에서 계속).....

● 수정 예시

1. Document Title Extraction (1번) 다듬기
(예: 줄바꿈/공백 처리 등의 정규화)

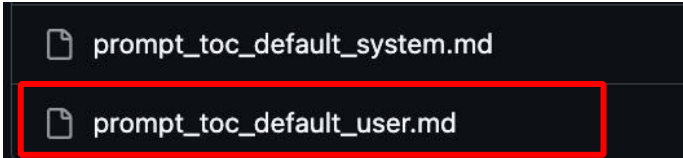
● 수정 불가:

1. {{raw_text}} 제거시 문서 정보 누락

전처리기 프로젝트 소스 사용 방법

- 테스트 설정

- **Enrichment 프롬프트 수정 안내 — (2) user_prompt 수정**
'preprocessor/resource/ **default.md' - Enrichment의 Prompt 파일



(Toc 추출 관련 사용자 프롬프트)

2. **Structural Marker Identification:** Scan through the document and quote all the key structural markers you find, such as:

○ Formal regulatory patterns: 제x장, 제x절, 제x관, 제x조

○ General section patterns: numbered headers (1., 2., etc.), lettered headers (가., 나., etc.)

○ Special sections: 부칙, 별지, 별표, etc.

3. **Systematic Section Extraction:** Work through the document from beginning to end, extracting each structural element in order:

○ For each main section, quote the exact title as it appears

○ For each subsection, quote the exact title and note which main section it belongs under

○ For each article/item, quote the exact title and note its parent section

○ Include any appendices, attachments, and addenda

4. **Hierarchy Building:** For each extracted element, explicitly note:

○ What level it should be at (main section, subsection, sub-subsection, etc.)

○ What its parent section is (if any)

○ What numbering it should receive in the final TOC (1., 1.1., 1.1.1., etc.)

5. **Structure Verification:** Review your extracted elements to ensure:

○ All structural elements are captured in document order

○ The hierarchy makes logical sense

○ No elements are duplicated or missed

- 수정 예시
1. **Structural Marker (2번)** : 다른 패턴 추가
(예: ①②③ 등의 원문자 항, I , II 등의 로마 숫자 등)

2. **Hierarchy (4번)**: 계층 매핑 명시
(예: 장→1, 절→2, 조→3) ※ 최대 4단계까지만

3. **그 외(3,5번)**: 추출 범위(부칙·별표 포함 여부)·검증 기준 조정 가능

(Toc 추출 관련 사용자 프롬프트)

Output Requirements

After your analysis, generate the table of contents with this exact format:

<toc>

TITLE:<document title>

1. <first main section title>

1.1. <first subsection title>

1.1.1. <first sub-subsection title>

1.2. <second subsection title>

2. <second main section title>

2.1. <subsection under second main section>

3. <third main section title>

</toc>

Formatting Guidelines

Start with TITLE: followed by the document title

Use hierarchical decimal numbering (1, 1.1, 1.1.1, etc.)

Follow each number with a space and the original title exactly as it appears

Maintain the document's logical hierarchy

Include appendices, attachments, and addenda as separate top-level items

Extract titles exactly as they appear - do not include explanatory content

Handle both formal regulatory structures and general section headers

Wrap the entire table of contents in <toc></toc> tags

- 수정 불가 (출력 형식에 영향을 줌)
- **Output Requirements**

- <toc>, TITLE, 1.1.1 번호 등

- **Formatting Guidelines**

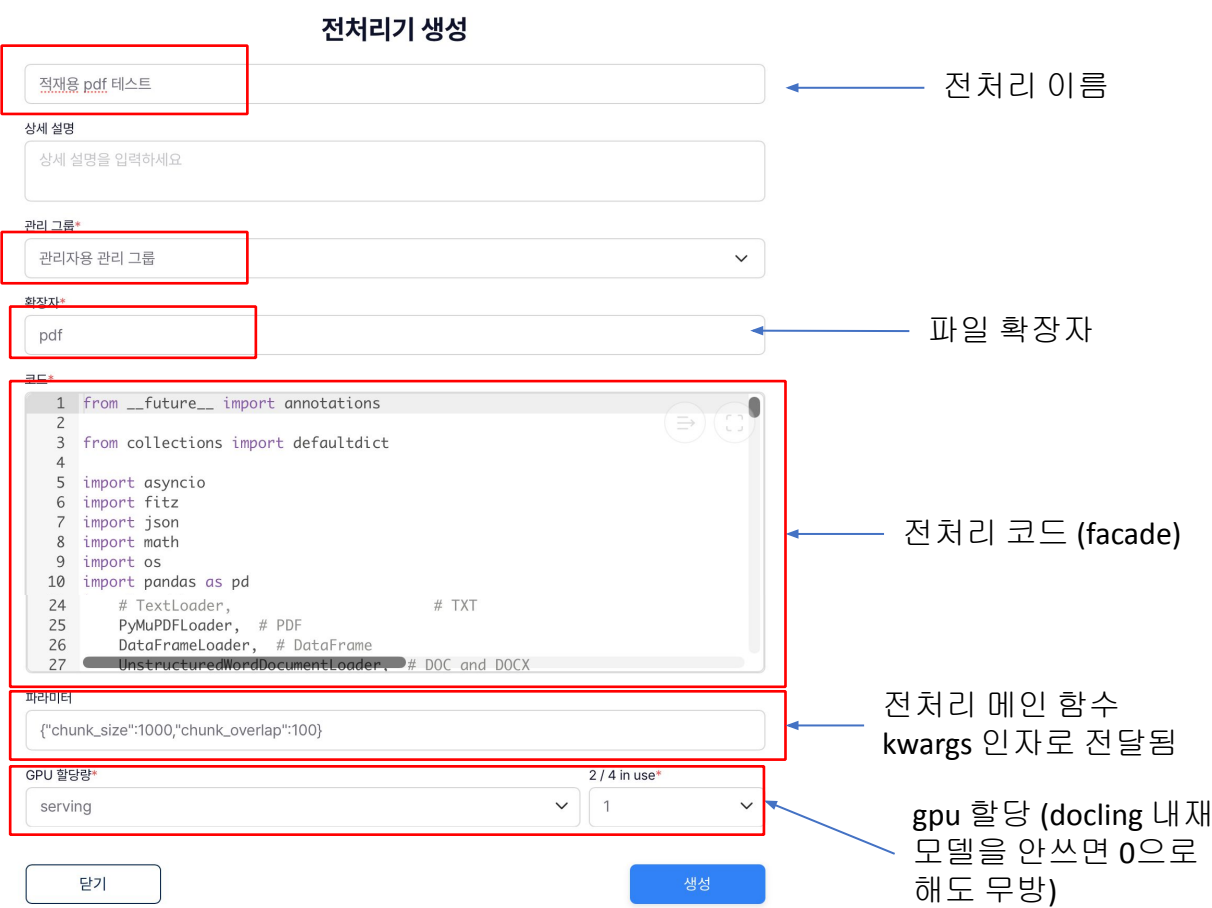
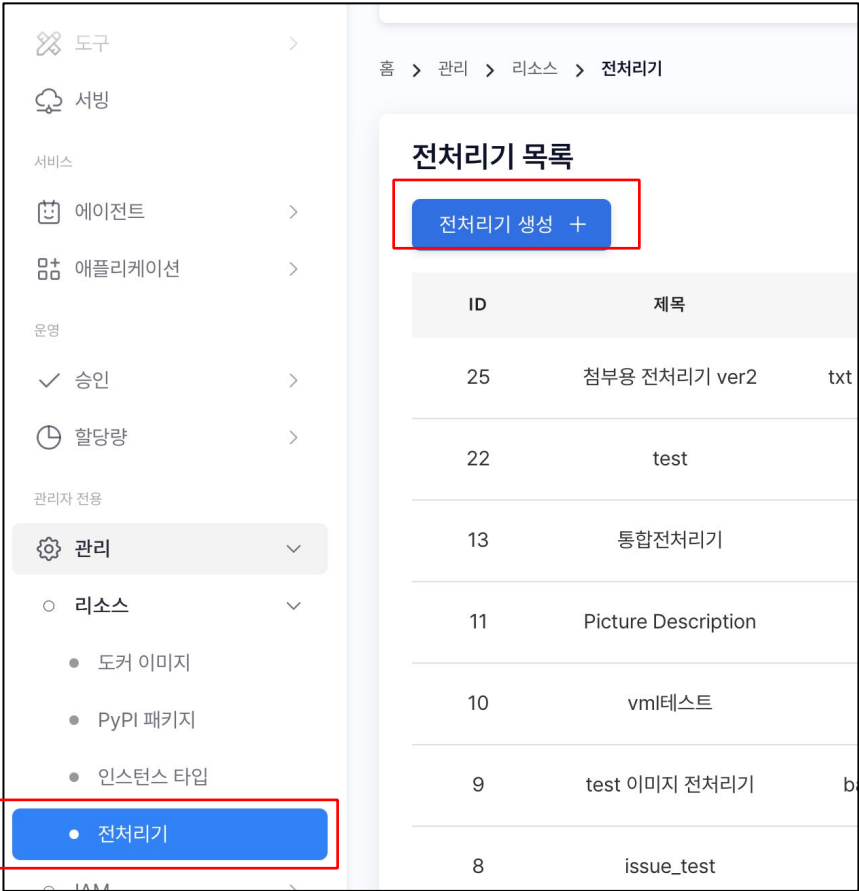
4. GenOS 전처리 사용방법

- **전처리기 등록 방법**
 - 전처리기 생성
 - YAML 파일 등록
 - 전처리기 배포
- **벡터 DB 등록 방법**
 - 벡터 DB 생성
 - 문서 추가 방법
- **오류 발생시 로그 확인 방법**
 - GenOS 전처리 로그
 - K9s 로그

GenOS 전처리 사용 방법

- 전처리기 등록 방법

- 전처리기 생성





GenOS 전처리 사용 방법

- 전처리기 등록 방법

- 전처리기 배포

전처리기 상세

기본 정보

배포 이력

컨테이너 상태

배포

배포 이력

ID

인스턴스 타입

도커 이미지

복제본

배포자

배포일시

중지자

중지일시

해당 정보가 없습니다

전체 0개 중 1 ~ 0 표시 / 페이지 표시 행수: 10

전처리기 배포

도커 이미지*

mnc/doc-parser-preprocessor

인스턴스 타입*

cpu core:4 Core 16 Gib

복제본*

1

닫기

생성

도커 이미지*

도커 이미지를 선택하세요

mnc/doc-parser-preprocessor:2.2.0-final

mnc/doc-parser-preprocessor:2.2.0-gpu-standard-final

전처리기 2.2.0 이상 지정

인스턴스는 2core 이상, 16gb 이상 선택

GenOS 전처리 사용 방법

- 전처리기 등록 방법

- 전처리기 배포

기본 정보

배포 이력

컨테이너 상태

이벤트 로그

컨테이너 로그

날짜	시간	상태	상세 설명
2025.11.06	12:34:45	배포 완료	메인 프로세스 실행 상태
	12:34:15	실행 준비 중	컨테이너 초기화 및 설정 중
	12:33:44	배포 중	배포 중 상태

정상적으로 등록되면 배포 완료로 표시됨.

날짜	시간	상태	상세 설명
2025.11.06	15:52:15	할당 대기 중	컨테이너 스케줄링 중
	15:51:43	배포 중	배포 중 상태
	15:49:58	미배포	배포 중이 아닌 상태

만약 할당 대기중으로 나오면 지정한 인스턴스가 부족한 상황이므로 다른 인스턴스를 선택해야 함.

GenOS 전처리 사용 방법

- 벡터 DB 등록 방법

- 벡터 DB 생성



데이터 형식은 문서로 지정



벡터 DB 생성

제목*
적재용 pdf 벡터db 테스트

상세 설명
상세 설명을 입력하세요

관리 그룹*
관리자용 관리 그룹

데이터 형식*
☒ 문서 ☐ QA (Deprecated) ☐ 자동적재 ☐ FAQ

수동 스키마 ☐

닫기

생성

GenOS 전처리 사용 방법

- 벡터 DB 등록 방법

- 문서 추가 방법

벡터 DB 상세

기본 정보

문서 목록

공유

업로드 +

삭제

파일 이름

검색어를 입력해 주세요

☐	ID	파일 이름	상세 설명	임베딩 서빙	벡터 개수	제작자	등록일시
해당 정보가 없습니다							

전체 0개 중 1 ~ 0 표시 / 페이지 표시 행수: 10

페이지 입력

문서목록 탭 --> 업로드 버튼

문서 추가

1 단계

2 단계

잘 정리된 문서일수록 채팅이 좋은 성능을 발휘합니다.
문서 작성 가이드와 문서 유형별 주의사항을 반드시 확인해주세요.

임베딩 서빙*

배치 크기*

bge-m3 (Rev ID=1)

64

문서 유형*

☐ 첨부용 전처리기 ver2
txt 한글 깨짐, pptx 처리 해결된 코드
[docx, pdf, ppt, pptx, hwp, hwp, csv, xlsx, txt, json, md]

☐ 1.3.0 Test
[pdf]

☒ 적재용 pdf 테스트
[pdf]

파라미터

Chunk Size

Chunk Overlap

1000

100

상세 설명

상세 설명을 입력하세요

닫기

이전

다음

임베딩 서빙 선택
문서유형에서 앞단계에서 등록한 전처리기 선택
다음 버튼

GenOS 전처리 사용 방법

- 벡터 DB 등록 방법

- 문서 추가 방법

문서 추가

1 단계

2 단계

파일 업로드*

여기에 파일을 놓거나 클릭하세요
(.pdf)

너도나도 AI 말로만 AI 외치는 _AI 위싱_ 주의보.pdf | 635.8 KB | 3.8 Mbps | 100% 완료

World Development Indicators 2004로 본 세계속의 한국경제 (2002).pdf | 192.0 KB | 14.1 Mbps | 100% 완료

닫기

← 이전

생성

등록하려는 pdf 파일 선택.
1개 이상 선택가능

파일 이름	상세 설명	임베딩 서빙	벡터 개수	제작자	등록일시	다운로드	상태
Development Indicators 2004로 본 세계속의 한국경제...		bge-m3	0	김성현 sunghun.kim	2025-11-06 16:09:38	↓	실행 중
AI 말로만 AI 외치는 _AI 위싱_ 주의보.pdf		bge-m3	0	김성현 sunghun.kim	2025-11-06 16:09:38	↓	실행 중

등록된 파일은 실행중에서 완료로 변경됨.

해당 파일을 선택하면 결과를 확인가능

너도나도 AI 말로만 AI 외치는 _AI 위싱_ 주의보.pdf

백스 슬기기

2 / 9

다음 페이지

KB 지식 비타민

SECTION HEADER

LIST ITEM

AI Washing이란 기업이나 제품이 실제로는 인공지능 기술을 사용하지 않으면서 광범위하게 활용하고 있는 것처럼 홍보하는 행위로 AI 열풍을 이용하는 마케팅 전략의 일환

LIST ITEM

이미 많은 기업이 친환경적인 것처럼 홍보하는 그린워싱(Greenwashing) 행태와 유사

LIST ITEM

기술 사용률 강조함으로써 투자자에게 매력적인 투자 대상으로 인식되어 더 많은 자본을 유치할 수 있으며, 보다 혁신적이고 기술 선도적인 이미지를 구축하여 비즈니스에서 경쟁사보다 유리한 위치를 점함으로써 기업 가치의 상승 효과를 기대할 수 있음

LIST ITEM

AI 위싱은 적절한 투자 자원의 배분 저해, 소비자 신뢰 저하, 과도한 기대감 유발 등 부정적인 결과를 초래할 수 있음

LIST ITEM

위싱의 유형은 다양하나, 자사 제품이나 서비스가 출시되는 과정에서 AI 기여도가 불분명함에도 광고나 홍보에서 AI 기술 사용을 강조하며 소비자를 기만하는 행위가 대표적

LIST ITEM

무인 매장 아마존고(Amazon Go)의 '저스트 워크 아웃(Just Walk Out)' 본 고객 픽업 지 자동적으로 결제가 청구되는 시스템으로 알려졌으나, 사실은 인도 지사 직원들이 수동으로 검토하는 것으로 밝혀짐

LIST ITEM

8 스타트업 준코(Joonko)는 AI를 기반으로 기업에 적합한 지원자를 추천한다는 허위 정보를 유도에 투자 자금을 유치했다는 이유로 미국 증권거래위원회(SEC)와 법무부로부터 기소됨

LIST ITEM

글로벌프렐릭션스 투자자문사 델파이(Delphia)와 글로벌프렐릭션스(Global Predictions)는 AI 위싱 행위로 미국 증권거래위원회로부터 각각 22만 5천 달러와 17만 5천 달러의 벌금이 부과됨

링크 3개 표시 중 (총 13개)

HEADER: <요약>
...
글자 수 793 자 · 페이지 1 ~ 2 · 🔍 체크 위치 찾기

HEADER: <요약>
<요약>...
글자 수 702 자 · 페이지 2 · 🔍 체크 위치 찾기

HEADER: <요약>, AI 위싱의 주요 특징과 부작용, [AI 위싱을 하는 이유]
<요약>...
글자 수 826 자 · 페이지 2 ~ 3 · 🔍 체크 위치 찾기

- 오류 발생시 로그 확인 방법

Development Indicators	bge-m3	0	김성현 sunghun.kim	2025-11-06 16:09:38	↓	오류 발생
계속의 한국경제...						

- GenOS 전처리 로그

[illegible]

전처리기의 컨테이너 상태 탭을 선택 후 컨테이너 로그를 선택

전처리기 상세

기본 정보 배포 이력 **컨테이너 상태**

이벤트 로그 **컨테이너 로그**

```
INFO: 2025-11-06 16:17:39 KST|[main.py:58 - run() ] Success: "/nfs-root/docs/Document-31814971.pdf"
INFO: 2025-11-06 16:17:39 KST|[main.py:69 - run() ] End: "/nfs-root/docs/Document-31814971.pdf" (76.25 secon
ds)
127.0.0.6:37106 - "POST /run HTTP/1.1" 200

INFO: 2025-11-06 16:18:06 KST|[main.py:56 - run() ] Start: "/nfs-root/docs/Document-31814971.pdf"

Token indices sequence length is longer than the specified maximum sequence length for this model (2347 > 512). Running this
sequence through the model will result in indexing errors

results=[]
```

GenOS 전처리 사용 방법

- 오류 발생시 로그 확인 방법

- k9s 로그

서버로 ssh 접속

k9s 실행

“/pre” 입력 해서 전처리기 deploy 탐색

genos 에 등록된 전처리기 id와 동일한 항목을 선택

“l” (L 소문자) 키를 눌러서 로그 내용 확인

```
Context: kubernetes-admin@kubernetes [RW]
Cluster: kubernetes
User: kubernetes-admin
K9s Rev: v0.50.16
K8s Rev: v1.27.16
CPU: 15%
MEM: 64%

deployments(llmops) [4]
NAME+READY
llmops-preprocess-api-deployment 1/1
preprocessor-25 1/1
preprocessor-91 1/1
preprocessor-94 1/1
```

```
Context: kubernetes-admin@kubernetes [RW]
Cluster: kubernetes
User: kubernetes-admin
K9s Rev: v0.50.16
K8s Rev: v1.27.16
CPU: 18%
MEM: 64%
```

```
----- pods(llmops/preprocessor-94) [1]
NAME                                PF  READY  STATUS    RESTARTS  CPU  %CPU/R  %CP
preprocessor-94-66db68d46f-mpmww  2/2  Running
```

```
Context: kubernetes-admin@kubernetes [RM]
Cluster: kubernetes
User: kubernetes-admin
K8s Rev: v0.56.16
K8s Rev: v1.27.16
CPU: 13%
MEM: 64%

Log: [lmpops/preprocessor-94-6db6b646f-9pmw:preprocessor-94] [tail]
AllContainers: Autoscaler[3m] FailSafe[3m] LimitTime: Wrap[3m]

preprocessor-94 result=[]
preprocessor-94
preprocessor-94
preprocessor-94
preprocessor-94 result=[]
preprocessor-94
preprocessor-94
preprocessor-94
preprocessor-94
preprocessor-94 result=[]
preprocessor-94
preprocessor-94
preprocessor-94
preprocessor-94 result=[]
preprocessor-94
preprocessor-94
preprocessor-94 result=[]
preprocessor-94
preprocessor-94
preprocessor-94 result=[None]
preprocessor-94
preprocessor-94 DINFO: 2025-11-06 16:19:16 KST [main.py:58 -
preprocessor-94 DINFO: 2025-11-06 16:19:16 KST [main.py:69 -
preprocessor-94 117.8.6.6:3001] --POST /api/HTTP/2.1 200
run() ] Success: "nfs-root/docs/document-31814971."
run() ] End: "nfs-root/docs/document-31814971.pdf"
```

전처리기 목록

전처리기 생성 +

ID	제목
94	적재용 pdf 테스트

5. 기타

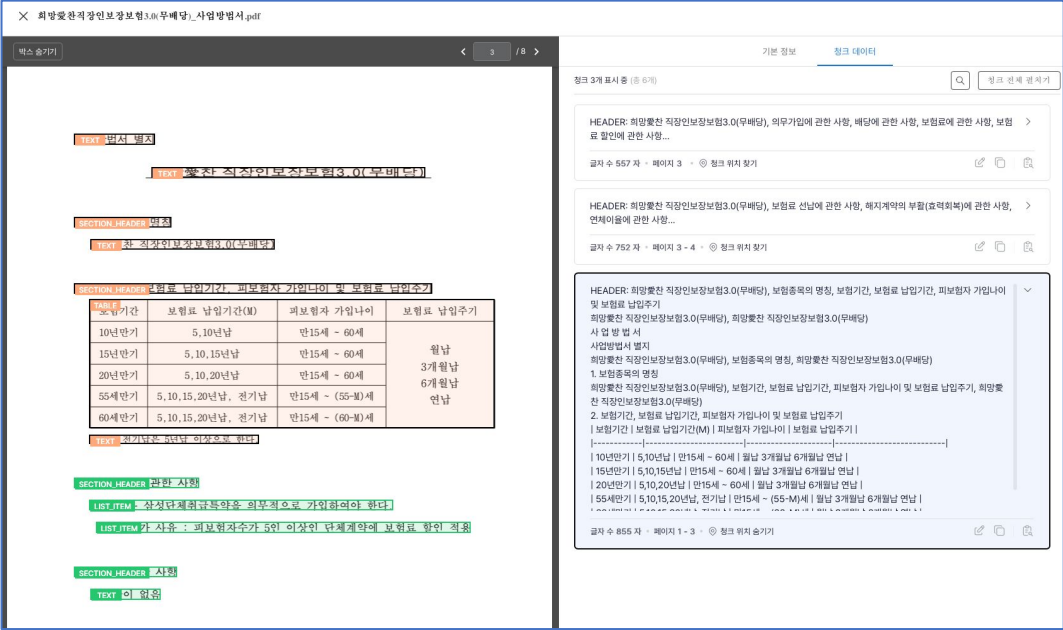
- **Facade로 할 수 있는것**
 - Facade 작업 예시
 - Facade 종류
 - 전처리기 배포
- **사이트에서 최적화 해야하는 포인트**
 - 전처리기의 목적
 - 적재후 기본 점검 예시 (GenOS 벡터 DB 결과 확인)
 - 청킹 예시

Facade로 할 수 있는 것

- Facade 작업 예시
 - 벡터DB데이터에 메타데이터 추가
 - OCR수행 유무 설정
 - PDF 외 다른 포맷 문서 처리 (hwp 등)
 - Enrichment 프롬프트 수정을 통한, 사이트에 최적화된 정보 추출
 - 청킹 알고리즘 수정 등
- [Facade 종류](#)
 - 첨부용 전처리기
 - 변종 - 변환용 전처리기
 - 적재용 전처리기
 - 소스파일 경로: doc_parser/genon/preprocessor/facade/

사이트에서 최적화해야 하는 포인트

- 전처리기의 목적
 - RAG 검색률을 높이는 것
 - 검색이 잘 되기 위해서는 문서의 텍스트가 적절히 청킹되어서 벡터 DB에 적재되어야 함.
- 적재 후 기본 점검 예시 (GeoOS 벡터 DB 결과 확인)
 - 텍스트가 잘 출력되었나?
 - 누락되는 텍스트는 없는가?
 - 텍스트가 깨져있지 않은가?
 - 청킹이 적절히 잘되고 있는가? (지능형 전처리기)
 - 청크가 너무 크거나, 문맥을 잘 유지하는가?
 - 섹션헤더가 적절히 잘 태깅되었나?



처리방안 예시

OCR 기능이 포함된 facade 코드를 이용
(yaml에서의 ocr=true 설정)

- 청킹코드 개선 (AI Search 파트에 문의)
- **Enrichment** 과정 최적화 (본 문서 33~39 pg) 를 통해 해결이 가능한지 테스트

사이트에서 최적화해야 하는 포인트

- 청킹 예시

✕ 한국은행금융결제망 IT업무 운영절차_20240125_전산정보국_20240125.pdf

백스 표시

< 3 / 13 >

⑤ 참가기관은 서버 IP 주소 및 서비스 포트(Service Port)를 처음으로 등록하거나 그 등록을 변경·삭제하고자 하는 경우 한은금융망 접속 서버 등록신청서 <별지 제2의2호 서식>를 등록희망일부터 3영업일 전까지 한국은행 IT전략국장에게 제출하여야 한다.

제6조(접속 해지) 한국은행 IT전략국장은 참가기관이 한은금융망 탈퇴, 그 밖의 사유로 한은금융망을 이용하지 않게 된 경우 다음 각 호의 조치를 하여야 한다.

- 통신망 접속 장비 설정내용 삭제
- 보안장비 설정내용 삭제

제3장 한은금융망 이용

제7조(단말기 접속 업무) ① 단말기 접속에 의한 업무처리는 한국은행이 정한 방식에 따라야 한다.

② 참가기관은 1대 이상의 단말기를 당해 참가기관의 대표단말기로 한국은행에 등록하여야 하며 한은금융망 운영시간 중에는 동 단말기를 한은금융망에 연결해 두어야 한다.

③ 참가기관은 1대 이상의 재해복구용 단말기를 등록할 의무가 있다.

④ 참가기관은 한은금융망 운영시간 중에 수시로 한은금융망의 공지사항에 게시된 내용을 열람하여야 한다.

제8조(서버 접속 업무) ① 서버 접속에 의한 업무처리에는 한국은행이 정한 방식에 따라야 한다.

② 서버 접속에 의한 업무처리를 변경하고자 하는 참가기관은 한국은행과 사전에 시스템 테스트를 실시하여야 한다.

③ 시스템 테스트를 실시하고자 하는 참가기관은 테스트 희망일 10영업일 전까지 한국은행 금융업무실장에게 테스트를 요청하여야 한다.

④ 한국은행 IT전략국장과 금융업무실장은 제3항에 따른 요청에 대하여 협의하여 테스트를 실시한다.

제4장 장애복구 대책

제9조(장애 또는 재해 처리절차) ① 참가기관은 참가기관의 접속장비, 통신회선 등의 장애 또는 재해로 인하여 정상적인 업무처리가 불가능하다고 판단될 경우 이를 지체 없이 한국은행 IT전략국장에게 통보하여야 한다.

② 한국은행 IT전략국장은 제1항의 통보를 받거나 한은금융망 관련 기기의 중대한 장애 또는 재해로 인하여 정상적인 업무처리가 불가능하다고 판단될 경우

<원본>

✕ 한국은행금융결제망 IT업무 운영절차_20240125_전산정보국_20240125.pdf

백스 숨기기

< 3 / 13 >

⑤ 참가기관은 서버 IP 주소 및 서비스 포트(Service Port)를 처음으로 등록하거나 그 등록을 변경·삭제하고자 하는 경우 한은금융망 접속 서버 등록신청서 <별지 제2의2호 서식>를 등록희망일부터 3영업일 전까지 한국은행 IT전략국장에게 제출하여야 한다.

제6조(접속 해지) 한국은행 IT전략국장은 참가기관이 한은금융망 탈퇴, 그 밖의 사유로 한은금융망을 이용하지 않게 된 경우 다음 각 호의 조치를 하여야 한다.

- 통신망 접속 장비 설정내용 삭제

SECTION_HEADER 3

제7조(단말기 접속 업무) ① 단말기 접속에 의한 업무처리에는 한국은행이 정한 방식에 따라야 한다.

② 참가기관은 1대 이상의 단말기를 당해 참가기관의 대표단말기로 한국은행에 등록하여야 하며 한은금융망 운영시간 중에는 동 단말기를 한은금융망에 연결해 두어야 한다.

③ 참가기관은 1대 이상의 재해복구용 단말기를 등록할 의무가 있다.

④ 참가기관은 한은금융망 운영시간 중에 수시로 한은금융망의 공지사항에 게시된 내용을 열람하여야 한다.

제8조(서버 접속 업무) ① 서버 접속에 의한 업무처리에는 한국은행이 정한 방식에 따라야 한다.

② 서버 접속에 의한 업무처리를 변경하고자 하는 참가기관은 한국은행과 사전에 시스템 테스트를 실시하여야 한다.

③ 시스템 테스트를 실시하고자 하는 참가기관은 테스트 희망일 10영업일 전까지 한국은행 금융업무실장에게 테스트를 요청하여야 한다.

④ 한국은행 IT전략국장과 금융업무실장은 제3항에 따른 요청에 대하여 협의하여 테스트를 실시한다.

SECTION_HEADER 4

제9조(장애 또는 재해 처리절차) ① 참가기관은 참가기관의 접속장비, 통신회선 등의 장애 또는 재해로 인하여 정상적인 업무처리가 불가능하다고 판단될 경우 이를 지체 없이 한국은행 IT전략국장에게 통보하여야 한다.

② 한국은행 IT전략국장은 제1항의 통보를 받거나 한은금융망 관련 기기의 중대한 장애 또는 재해로 인하여 정상적인 업무처리가 불가능하다고 판단될 경우

<잘된 청킹>

→ 섹션 헤더를 기준으로 올바르게 청킹.

✕ 한국은행금융결제망 IT업무 운영절차_20240125_전산정보국_20240125.pdf

백스 숨기기

< 3 / 13 >

⑤ 참가기관은 서버 IP 주소 및 서비스 포트(Service Port)를 처음으로 등록하거나 그 등록을 변경·삭제하고자 하는 경우 한은금융망 접속 서버 등록신청서 <별지 제2의2호 서식>를 등록희망일부터 3영업일 전까지 한국은행 IT전략국장에게 제출하여야 한다.

제6조(접속 해지) 한국은행 IT전략국장은 참가기관이 한은금융망 탈퇴, 그 밖의 사유로 한은금융망을 이용하지 않게 된 경우 다음 각 호의 조치를 하여야 한다.

- 통신망 접속 장비 설정내용 삭제

SECTION_HEADER 3

제7조(단말기 접속 업무) ① 단말기 접속에 의한 업무처리에는 한국은행이 정한 방식에 따라야 한다.

② 참가기관은 1대 이상의 단말기를 당해 참가기관의 대표단말기로 한국은행에 등록하여야 하며 한은금융망 운영시간 중에는 동 단말기를 한은금융망에 연결해 두어야 한다.

③ 참가기관은 1대 이상의 재해복구용 단말기를 등록할 의무가 있다.

④ 참가기관은 한은금융망 운영시간 중에 수시로 한은금융망의 공지사항에 게시된 내용을 열람하여야 한다.

제8조(서버 접속 업무) ① 서버 접속에 의한 업무처리에는 한국은행이 정한 방식에 따라야 한다.

② 서버 접속에 의한 업무처리를 변경하고자 하는 참가기관은 한국은행과 사전에 시스템 테스트를 실시하여야 한다.

③ 시스템 테스트를 실시하고자 하는 참가기관은 테스트 희망일 10영업일 전까지 한국은행 금융업무실장에게 테스트를 요청하여야 한다.

④ 한국은행 IT전략국장과 금융업무실장은 제3항에 따른 요청에 대하여 협의하여 테스트를 실시한다.

SECTION_HEADER 4

제9조(장애 또는 재해 처리절차) ① 참가기관은 참가기관의 접속장비, 통신회선 등의 장애 또는 재해로 인하여 정상적인 업무처리가 불가능하다고 판단될 경우 이를 지체 없이 한국은행 IT전략국장에게 통보하여야 한다.

② 한국은행 IT전략국장은 제1항의 통보를 받거나 한은금융망 관련 기기의 중대한 장애 또는 재해로 인하여 정상적인 업무처리가 불가능하다고 판단될 경우

<잘못된 청킹>

→ 섹션 헤더와 무관한 청킹